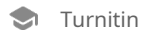


GVI mapper FYP 3 updated (1).docx



Document Details

Submission ID

trn:oid:::3618:103408219

Submission Date

Jul 4, 2025, 5:47 AM GMT+5

Download Date

Jul 4, 2025, 5:50 AM GMT+5

File Name

GVI mapper FYP 3 updated (1).docx

File Size

1.5 MB

84 Pages**14,558 Words****82,333 Characters**

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **1 AI-generated only 0%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



GVI Mapper

Abdus Sami Chheena (210617)

Maria Altaf (210464)

Muhammad Umar Riaz (210648)



Spring - 2024

Supervised By

Tahir Akram

Department of Creative Technologies

Faculty of Computing & AI

Air University, Islamabad

PROJECT REPORT



Version

V X.0

NUMBER OF
MEMBERS

3

TITLE

SUPERVISOR NAME

Tahir Akram

MEMBER NAME

REG. NO.

EMAIL ADDRESS

Abdus Sami Chheena

210617

210617@students.au.edu.pk

Maria Altaf

210646

210646@students.au.edu.pk

Muhammad Umar Riaz

210648

210648@students.au.edu.pk

MEMBERS' SIGNATURES

Supervisor's Signature

Note 1: This paper must be signed by your supervisor

Note 2: The soft-copies of your project report, source codes, schematics, and executable should be delivered in a CD

APPROVAL CERTIFICATE

This project, entitled as “GVI MAPPER” has been approved for the award of

Bachelors of Science in Artificial Intelligence

Committee Signatures:

Supervisor:

(Mr. First Name Surname)

Program Lead:

Head of Department:

(Dr. Imran Ihsan)

DECLARATION

I/We, hereby, declare that “No portion of the work referred to, in this project has been submitted in support of an application for another degree or qualification of this or any other university/institute or other institution of learning”. It is further declared that this undergraduate project, neither as a whole nor as a part thereof has been copied out from any sources, wherever references have been provided.

MEMBERS' SIGNATURES

ACKNOWLEDGEMENTS

It is usual to thank those individuals who have provided particularly useful assistance, technical or otherwise, during your project. Your supervisor will obviously be pleased to be acknowledged as he or she will have invested quite a lot of time overseeing your progress.

DEDICATION

This is an optional section.

In this section you dedicate your project to anybody that you feel motivates you for hard work and putting effort for successful life.

Executive Summary

This should be no more than one page in length (200 words approx.). The summary should allow the reader who is unfamiliar with the work to gain a swift and accurate impression of what the project is about, how it arose and what has been achieved.

It is recommended, you write this section when the report is finished.

Table of Contents

This should give a complete list of what the report contains starting with the abstract (the title page is not included in the contents list).

Contents

Chapter 1	1
Introduction	1
1.1. Project Introduction	1
1.2. Existing Examples / Solutions	2
1.3. Business Scope	3
1.4. Useful Tools and Technologies	5
1.5. Project Work Break Down	6
1.6. Project Feasibility	7
1.6.1. Technical Feasibility	8
1.6.2. Operational Feasibility	8
1.6.3. Economic Feasibility	9
1.6.4. Schedule Feasibility	11
1.6.5. Specification Feasibility	12
1.6.6. Information Feasibility	12
1.6.7. Motivational Feasibility	12
1.6.8. Legal & Ethical Feasibility	13
1.7. Risk List	13
Chapter 2	16
Requirement Specification and Analysis	16
Requirement Specification	16
2.1. Functional Requirements	16
2.2. Non-Functional Requirements	17
2.3. System Use Case Modeling	17
2.4. Fully-dressed Use Cases	18
	8

2.5. Domain Model	27
Chapter 3	29
System Design.....	29
3.1. System Architecture	29
3.2. Workflow Design	30
3.3. UML diagrams	31
3.4. Dataflow and preprocessing pipelines	33
3.5. Activity Diagram	34
3.6. Algorithms and model design.....	35
3.7. Scalability and performance considerations	37
3.8. User Interface Design.....	39
Chapter 4	43
System Development	43
4.1. Coding Standards.....	43
4.2. Development Environment	44
Packages and Libraries Used:.....	44
Third-Party Libraries:.....	45
4.2.1. Hardware Requirements:	45
4.2.2. Software Requirements:	46
4.3. Dataset Acquisition and Preprocessing.....	47
4.4. Model Selection and Development	50
• Data Preparation:	51
• Model Training:	51
• Hyperparameter Tuning:.....	52
Early Stopping and Regularization:	52
Chapter 5	53
Evaluation of Proposed Pipeline	53
5.1. Evaluation Methodology	53
5.2. Evaluation Environment	55
	9

5.3. Evaluation Results	55
Chapter 6	57
Software Deployment.....	57
6.1. Installation / Deployment Process Description	57
6.1.1. Prerequisites	57
Operating System Compatibility.....	57
Hardware Requirements	57
Software Dependencies	57
Specific Driver Requirements	58
6.1.2. Obtaining the Software.....	58
Cloning from a Repository.....	58
Downloading a Release Package	58
6.1.3. Setting up the Environment.....	58
Creating a Virtual Environment.....	58
Installing Dependencies.....	59
(Optional) Configuring GPU Support	59
6.1.4. Configuration.....	59
Configuration Files	59
API Keys or Credentials	59
Setting Environment Variables.....	60
6.1.5. Running the Software.....	60
Command-Line Interface (CLI) Usage.....	60
Graphical User Interface (GUI) Launch	60
Web Interface Deployment	60
6.1.6. Verification of Installation	61
Running Sample Data	61
Checking Output Files and Logs.....	61
Basic Functionality Tests.....	61
6.2. System Requirements	61
	10

6.2.1. Minimum Requirements	61
6.2.2. Recommended Requirements	62
6.2.3. Optimal Requirements	62
6.2.4. Software Dependencies (Revisited with Specific Versions)	62
6.3. Troubleshooting	63
6.3.1. Common Installation Issues	63
6.3.2. Common Runtime Issues	63
6.3.3. Logging and Error Reporting	63
6.4. Deployment on Different Platforms (If Applicable)	63
6.4.1. Cloud Deployment (e.g., AWS, Google Cloud, Azure)	63
6.4.2. Edge Device Deployment (If Applicable)	64
6.5. Updates and Maintenance	64
6.5.1. Checking for Updates	64
6.5.2. Updating the Software	64
6.5.3. Maintenance Procedures	65
REPORT APPROVAL CERTIFICATE	66
References	67
Appendices	68
Appendix A – Dataset Samples	68
Appendix B – Camera Calibration Outputs	68
Appendix C – Additional Figures	68
Appendix D – Code Snippets	68
Appendix E – Evaluation Metrics	68

List of Figures

If the report contains figures or tables a list of these should be provided. The list should give the table or figure number, the title of the table or figure and the page number. If only a few tables and figures are present, they may be treated on one page. Remember that all figures and tables used must be referred in the text. For example, “The class diagram shown in Figure 2.1”

Figure1. 1: Work breakdown Structure

Figure 1.2: Sample Gant Chart

Figure 2.1: Sample Use case Diagram

Figure 2.2: System Sequence Diagram

Figure 2.3: Domain Model

Figure 3.1: Software Architecture Diagram

12

Figure 3.2: UML Class Diagram

Figure 3.3: Sequence Diagram

Figure 3.4: Entity Relationship Diagram

Figure 3.5: Database Schema

Figure 3.6: Common GUI elements

Figure 3.7: Example Login Page UI Design with description in text

LIST OF TABLES

Table 2.1: Functional Requirements

Table 2.2: Functional and Non-Functional Requirement

Table 2.1: Use Case 1

Table 2.2: Use Case 1

Table 3.1: Data Dictionary

This page is kept blank

Chapter 1

Introduction

We work in the area of construction of 3D scenes based on 2D information with the help of the method Monocular Depth Estimation (MDE). Such new direction has found great popularity because of its wide use in autonomous driving, navigation, 3D reconstruction. MDE could also be used to create AI contents and assist self-learning agents because it offers a strong visual data that is vital in description of an environment and understanding of a scene. Our project would assist 3D environment reconstruction and data labeling using MDE to provide usable information. Such insights allow the actors and agents to communicate with their environments efficiently and are able to carry out the actions they would like to execute without difficulties.

1.1. Project Introduction

This project will aim at generating 3D labeled data of an environment based on 2D data through the use of computer vision.

The solution will serve as a low-cost alternative to well-established 3D sensing technologies, SONAR and RADAR, because it uses easily accessible hardware, in this case, cameras. The goal of the project is to create high-resolution 3D renderings that can serve in such spheres as architecture, urban planning, self-driving cars, and so on, using computer vision.

The needs of targeting the project are based on the following industries and users. These are

- **Architecture & Construction Firms:** These organizations would be able to apply 3D reconstructions to design, planning and simulation of buildings.
- **Urban Planning Departments:** They will be able to take advantage by applying 3D maps in development of cities, infrastructure, and management of resources.
- **Real Estate Companies:** Properties have the ability to get 3D reconstructions which provide a more realistic experience to the potential building buyer.
- **Cultural Heritage Organizations:** The technology enables historical sites to be preserved and allows room to virtual tours of the sites using exhaustive 3D models.
- **Film & Video Game Studios:** Virtual sets and surroundings can be made with the help of 3D scene generation to create media.
- **Autonomous Vehicles Manufacturer:** MDE may be helpful in enhancing car navigation and mapping the environment of an autonomous vehicle.

- **Robotic Companies:** The 3D environmental information has the potential to help robots with their assistance in navigation and completion of tasks.
- **Healthcare Providers:** 3D mapping can assist in medical imaging, surgical planning, and environmental simulations for healthcare applications.
- **Surveying & Mapping Companies:** They can use the technology to create accurate and detailed 3D models for topographic and land surveys.
- **Environmental Agencies:** These organizations can monitor environmental changes and assess terrain for conservation and planning purposes.
- **Virtual & Augmented Reality Developers:** MDE will provide them with the 3D environments needed for immersive virtual and augmented experiences.
- **Forensics & Law Enforcement:** For crime scene reconstruction and analysis.

1.2. Existing Examples / Solutions

According to my research and survey, some companies are in the process of finding solutions through which they develop this rich data of the 3D environment. Such solutions are usually based on high-tech solutions like LiDAR, RADAR, and SONAR, or a combination of them with a computer vision. Though these companies have come a long way in producing correct 3D reconstructions, majority of the available solutions rely on cumbersome and costly hardware. This dependency jack-ups prices and restricts smaller organizations and individuals who come across it. Moreover, a lot of the systems are domain-specific, including autonomous driving or industrial usage, meaning that they lack flexibility to meet a variety of use cases. In the next section, I have discovered some of the companies operating in the area and their strategies and technologies.

Companies	Solutions	Shortcomings
Matterport	Creates digital twins of real-world spaces for virtual tours, real estate marketing, and facility management.	Primarily focuses on indoor spaces and requires specialized hardware for capturing, limiting its accessibility for widespread use.
Esri	Offers ArcGIS software for creating 3D models of cities and landscapes for urban planning and environmental analysis.	Complex software with a steep learning curve, often requiring specialized training to use effectively.
Bentley Systems	Provides infrastructure engineering software for creating detailed 3D models of buildings, roads, and utilities.	Mainly targets large-scale infrastructure projects, potentially overlooking smaller-scale environmental reconstruction needs.

Pix4D	Develops photogrammetry software for creating 3D models from drone imagery, used in construction, agriculture, and surveying.	Relies heavily on aerial imagery, which may not capture all details of complex environments, especially in densely built areas.
AutoDesk	Offers various software solutions for 3D modeling and visualization of real-world environments in architecture and engineering.	Their solutions often require significant manual input and expertise, limiting the speed and scalability of 3D environment creation.
Wayfair	Offers AR available to furnish items in homes of the customers since 2017. They also offer a "View in Room" option which gives the ability to the user to see how they can furnish their spaces and buy accordingly.	Limited to furniture and home decor items, lacking broader environmental reconstruction capabilities.
Osso VR	Provides a virtual reality platform for medical professionals, creating lifelike simulations of medical procedures and surgeries for training purposes.	Focused on specific medical scenarios rather than general environmental reconstruction.
Luxcarta	Develops 3D building reconstruction from LiDAR data for digital twin technology, urban planning, and simulations.	Relies heavily on LiDAR data availability and may struggle with complex or irregular building structures.
IGN	Collects LiDAR point cloud data for most of France's territory, enabling various 3D reconstruction applications.	Data collection is geographically limited and may require significant processing to derive useful insights.

1.3. Business Scope

The proposed business provides high business opportunities since it considers the need to find cheap and accurate solutions on 3D environmental mapping. The traditional technologies are also quite dependent on costly equipment like LiDAR, and RADAR, and this may limit accessibility by small companies and enterprises. Through the utilization of cheaper technologies using computer vision, this project aims at democratizing the use of 3D scene reconstruction technology and exposing to a wider audience. The project has a strong potential to undergo commercialization because of its orientation towards affordability and versatility. More resources and strategic partners will also contribute to its throughput and commerciality as a product.

Industries	Needs	Requirements
Construction & Architecture	Accurate 3D models of buildings and construction sites for planning and monitoring.	• GPU for quick processing • Ability to handle large scale environment • Integration with existing CAD software
Urban Planning	Detailed 3D labeled model for Urban development and designing planning and environmental analysis	• GPU for quick processing • Ability to handle large scale environment • Dataset for city labeling • 3D model simulation
Environmental Conservation	3D reconstruction of environment for study, monitoring and preserving the environment.	• GPU for quick processing • Ability to capture & process terrains in its 3D reconstruction • Labeling for all environmental factors for detailed 3D environment
Cultural Heritage Preservation	Digital Archiving of historical sites, artifacts and culture while also simulating some of the data	• GPU for quick processing • Ability to capture fine details of delicate surfaces
Automotive Industry	3D modeling for design prototyping and customization in luxury market	• GPU for quick processing • Precise scanning abilities of complex shapes and structures

		• Integration with CAD/CAM systems
Healthcare	3D reconstruction of anatomical structures for surgical planning and medical education	• Powerful GPU for quick processing • High-accuracy & quick scanning of organic shapes • Tools and data that provides details regarding the 3D modeled organ
Agriculture	3D mapping of crops and field for precision agriculture and phenotyping	• GPU for quick processing • Ability to capture large area efficiently • Integration of the system with agriculture management system
Media & Entertainment	Creation of realistic 3D and 2.5D environment for environment and virtual video productions on stages and theatres	• Powerful GPU for quick processing • Ability to convert 3D details to a desired 2D or 3D details • Ability to cover an actor's action with a desired simulated background
Forensics & Law Enforcement	3D reconstruction of crime scenes and accidental sites for investigation, discussion and courtroom presentation	• GPU for quick processing • Rapid on-site scanning capabilities • Integration with case management software

1.4. Useful Tools and Technologies

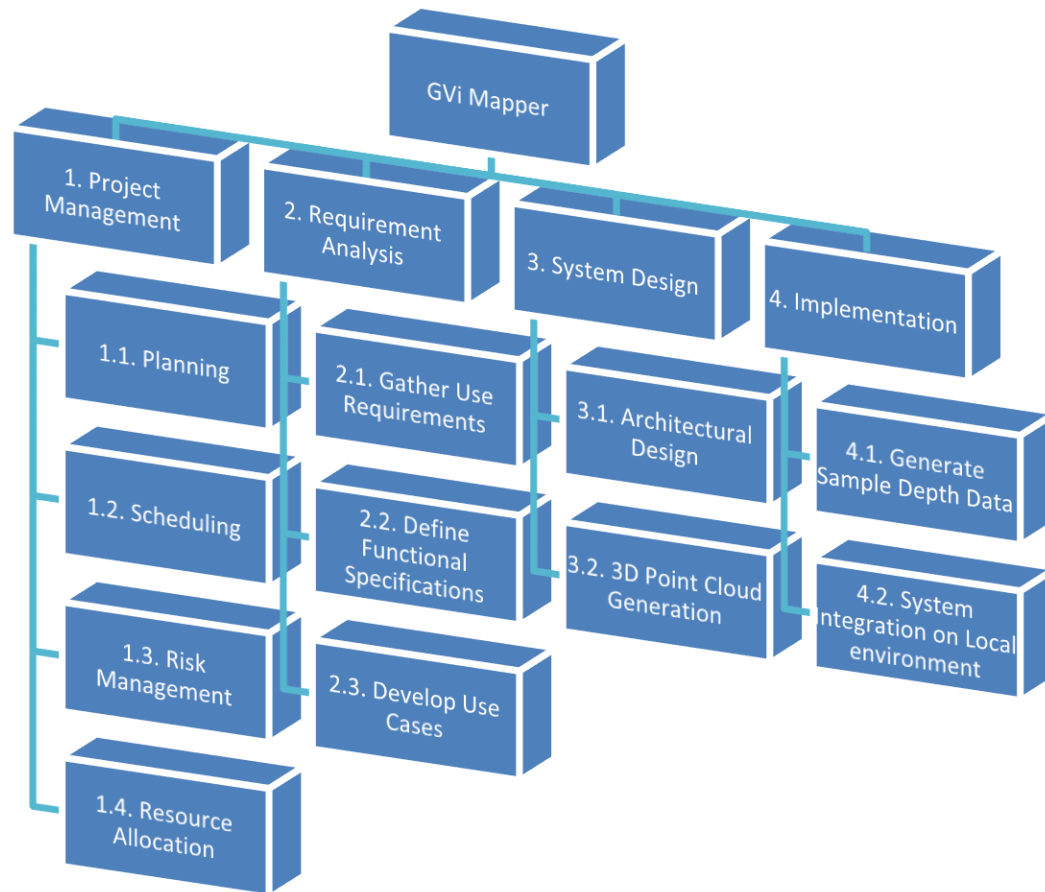
The development and implementation of this project require a combination of advanced tools and technologies to ensure efficiency and accuracy. The primary programming language chosen is Python due to its extensive libraries and frameworks for computer vision and deep learning, such as Numpy, OpenCV, Open3D and PyTorch. The development environment selected are Google Collab and Visual Studio Code or Anaconda. Google Collab is useful as it provides a developed environment with almost all required libraries pre-installed. Also there are all extensive supports in its installation as well as existing cloud-based GPU support. Anaconda and Visual Studio Code are useful Google Collab fails in providing a 3D simulation of the data

by providing an interactive and visual window. While the project does not require a database for the initial prototype, future iterations may incorporate a lightweight database such as SQLite to store 3D data and metadata for user access. The software will be compatible with cross-platform operating systems, including Windows, Linux, and macOS, to ensure broader accessibility. Additionally, no specific network protocol is implemented at this stage, as the project focuses on standalone processing; however, future extensions may include network protocols like REST APIs for data sharing and remote access.

1.5. Project Work Break Down

To achieve the objectives of this project, we have meticulously planned and divided the work into distinct phases, ensuring that each task aligns with the project timeline and goals. The allocation of tasks is based on individual team members' expertise and availability, with additional time dedicated to areas requiring further research or development. By focusing on efficient task management and resource allocation, we aim to deliver each component within the stipulated deadlines while addressing potential challenges. The following sections outline the tasks, their assignments, and the timeframes for completion.

Abdus Sami Chheena (210617)	Maria Altaf (210646)	Muhammad Umar Riaz
• Strength: • Strong Python Programming skills • Basic Advance Neural Network Architecture Understanding • Basic Research skills • Weakness: • Weak Managerial Skills • Weak Communication Skill	• Strength: • Strong Python Programming skills • Strong Documentation skills • Weakness: • Low Resilience • Slightly weak in Advance ANN architecture	• Strength: • Strong Python Programming skills • Strong Communication skills • Strong Prompt engineering skills • Basic 3D library Implementation skills • Weakness: • Slightly off-schedule



1.6. Project Feasibility

The first thing to lay down when commencing a project is to determine whether a project or a product is feasible or not. Feasibility has a definition of the extent to which suitable data and information are readily accessible or accessible using available means like personnel, skills, time and equipment. It is applied more or less as an indication of the utility or usefulness of the development of a software system to you (or organization). Such behavior is repeated during the life cycle.

There are many types of feasibilities:

2. Technical
3. Operational
4. Economic
5. Schedule
6. Specification
7. Information
8. Motivational
9. Legal and Ethical

1.6.1. Technical Feasibility

Technical possibility of this project is also well thought of so as to have developed it with the available technology and expertise (feasible tonnage and expertise). The project is based on the depth estimation algorithms and computer vision techniques alongside deep learning models, all those components are thoroughly advanced in the field, and can be supported with the help of library Open3D, OpenCV, Numpy and PyTorch.

The staff is well connected to programming, algorithm design and application of these frameworks, so the technical needs of the project will be implemented successfully. Also, high-performance computers and basic GPUs, which are the hardware resources, are sufficient to make the initial testing and implementation.

The set of possible challenges, including real-time optimization of the model or work with the big data set, are outlined but judged to be within the scope of knowledge and expertise of the team, as well as the iterative development process. According to these factors, the project can be called technically feasible within the determined scope.

1.6.2. Operational Feasibility

The feasibility of the project operations has been evaluated so as to provide the solution that can be easily applied in the project and be well used by the intended users. This is important because the project meets a vital problem of the 3D scene reconstruction with cost-efficient and accessible approaches, and it is an effective solution to the industries like architecture, autonomous systems, and urban planning.

The final users who will be professional and organizations in these fields will find the solution useful and practical in that it does not require the use of costly equipment such as the LiDAR and RADAR systems. The output and interface of the system will be user-friendly and it should incur the least technical knowhow to use it.

Moreover, the team possesses the resourcefulness to make and show the system in a good manner and the cycle of feedback with the possible consumers will help to cover the gaps in the work. It is on this basis that the project is found to be operationally feasible and wants good potential of acceptance by users appointed to use it.

1.6.3. Economic Feasibility

Evaluating the economic feasibility of this project involves a comprehensive analysis of both cost and benefit estimates to determine its financial viability.

Cost Estimates:

Development Costs (One-Time):

- **Personnel Expenses:** Salaries for team members during the development phase.
- **Software and Tools:** Licenses for any required proprietary software or development tools.
- **Hardware Resources:** Procurement of necessary hardware to support development activities.

Maintenance and Operation Costs (Ongoing):

- **System Updates:** Periodical update to be compatible and secure.
- **Technical Support:** This is the provision of support in the form of assistance to end-users when necessary.
- **Operational Overheads:** Fees that are related to hosting, utilities and other necessities of operation.

Benefit Estimates:

Tangible Benefits:

- **Cost Reduction:** By utilizing cost-effective methods for 3D scene reconstruction, the project reduces the need for expensive equipment, leading to significant savings.
- **Revenue Enhancement:** Offering an innovative solution can attract new clients and open additional revenue streams.

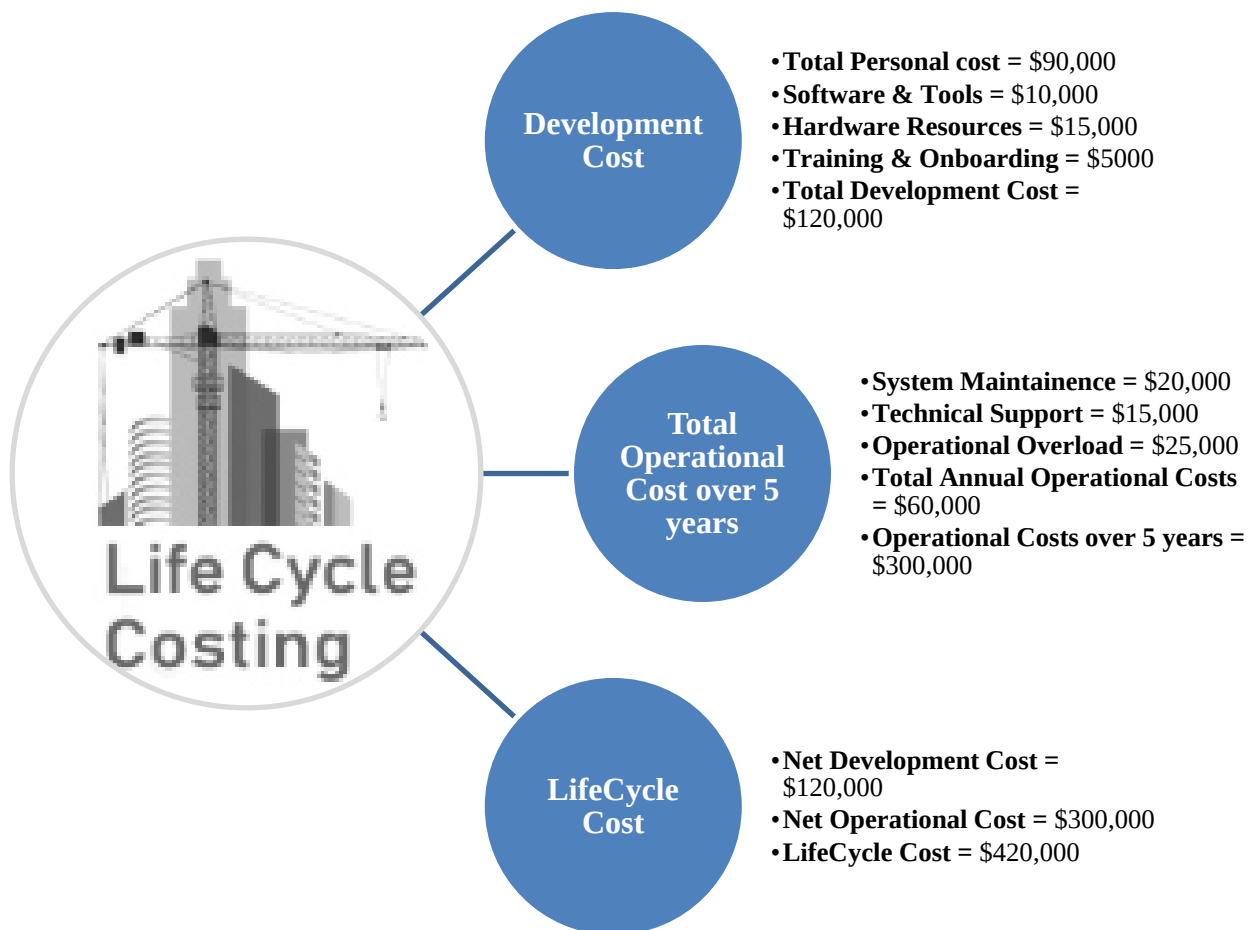
Intangible Benefits:

- **Improved Information Quality:** Increased precision and resolution of 3D data make it more helpful in decision-making.
- **Higher Job Satisfaction:** Ensuring that the tools offered are user friendly would also increase job satisfaction of the professionals using the system.

- **Increased External Ranking:** The use of refined technology has the potential to improve on the position of the organization in the industry.

With the help of reduction into detailed tasks taking the help of life cycle cost models we have come up with these estimations based on past experiences in similar projects. The measurement of the function point helps in the accuracy of our cost estimates further.

In conclusion, the anticipated benefits, both tangible and intangible, outweigh the projected costs, indicating that the project is economically feasible and poised to deliver substantial value to stakeholders.



1.6.4. Schedule Feasibility

The schedule of the project has been carefully planned after many discussions. The entire projects timeframe was carefully planned while considering all the factors such as budget, resources and team members.

1. Project Timeline:

- **Estimated Project Duration:** 18 months
 - Prototype development (3 Months)
 - Beta version release (6 Months)
 - Final Testing phase (6 Months)
 - Project Completion & Deployment (3 Months)

2. Resource Availability:

- **Development Team:** 3 full-time AI experts
- **3D Modeling specialist:** 1 full-time expert
- **Documents specialist:** 1 full-time expert
- **Project Manager:** 1 full-time expert
- **Quality Assurance Team:** 2 full-time tester

3. Task Breakdown & Estimation:

- **Task Breakdown Estimation:** 2 months
- **Core Algorithm Development:** 6 months
- **User Interface Design & Development:** 4 months
- **Integration & Testing:** 4 months
- **Optimization & Fine-tuning:** 2 months

4. Risk Assessment:

- **Potential delay in Algorithm:** Mitigated by allocating additional resources if needed.
- **Integration challenges with various data sources:** Addressed through early prototyping and iterative development.

5. Flexibility & Scalability:

- Agile development methodology adopted to allow for adjustments in project scope and timeline.
- Provision for additional resources if project demands increase

Based on our assessment, the project is deemed schedule feasible. The allocated time of 18 months is sufficient to complete all necessary tasks with the current team composition. Regular progress

monitoring and adherence to the agile methodology will ensure that we meet our deadlines and deliver a high-quality 3D reconstruction solution within the specified timeframe.

1.6.5. Specification Feasibility

In our project the requirements are well defined and have been articulated in collaborative with the stakeholders in form of workshops, thus clear and cannot be misunderstood as they mirror the needs of the users. Project Scope boundaries are thoroughly drawn to avoid capitalizing on scope creep and thus is restricted to be focused and reachable making use of the available resources and time constraints.

Through such methodologies as Specification by Example we were able to record requirements in a way that makes sense to all stakeholders in the form of realistic examples. This method will make the specifications to be clear and testable, and this will make the development process to have a strong background.

Following this critical examination, the project specifications have been found feasible since they are clear, doable and comply with the objectives and restrictions of the project.

1.6.6. Information Feasibility

We have critically evaluated the requirements of data to be used in our project after making sure that it fits in our model specification. In our datasets, the required details, quality and type of information has been presented that will allow us to generate precise and quality outputs.

We approach three of the main datasets, Hypersim, Kitty, and Vkiti2. Such datasets provide indoor and outdoor scene specification with considerable details containing vital information in terms of transparency, reflection, and fine visualization, and intricate environmental properties. These varied data help give a complete basis on which our model may produce stable 3D reconstructions.

Using these datasets we have no worries that the information we require is adequate and of good quality so that the model would be effective in producing results that would be accurate.

1.6.7. Motivational Feasibility

In order to make our plans motivational we have laid down the various project milestones, frequent meetings and a work environment where everyone works together to remain motivated and focused on work. Also, motivation and awards that will be given in place of milestones that are accomplished will serve to keep the enthusiasm levels high and a high commitment to the project.

Team motivation plays a key role in successful implementation of the project and the structures in the provinces aim at motivating the team with regular growth and effective task completion.

We are sure that, according to these factors, the team will remain motivated and will make a significant contribution to the success of the project.

1.6.8. Legal & Ethical Feasibility

Pre-trained models, employed in the project, are taken in the public repositories on GitHub. The models are open source under amavic core Apache 2.0 license that has a free right to use, modify, and also distribute. We have critically examined the licensing conditions of every model used to ascertain that we have observed the conditions stipulated in the licenses such as giving the due credit to the initial authors in line with the conditions stipulated in the licenses.

Although we have presented the models publicly, we also carry out an ethical review to ascertain that the models used do not present any ethical issues or bias when used. Datasets that are applied to train these models have been audited to demonstrate their fairness and we will be taking steps to reduce any possible bias in the outcomes. Also, since the models are built with the publicly available data, there is no breach of privacy regarding the usage of the data.

1.7. Risk List

Prior to an elaborate risk list, there is a need to identify and recognize the risk to be considered as detrimental to the achievement of the project. An evaluation of risks at the initial stages of any project enables us to develop effective mitigation strategies, and the project goes on without hitches. The list of the risks below shows the most important risks in the decreasing order of significance, as well as the mitigation or contingency plan. Such a list will constantly be tracked and updated in the course of the project life, which will enable us to overcome emerging challenges as they emerge in the project and deliver it timely.

- **Technical Complexity:**

Risk: The well-developed complexity of algorithms in 3D reconstruction can cause some untold difficulties and stoppages.

Mitigation: Invest more resources in R&D, do periodic technical inspections and retain flexibility of schedule.

- **Data Quality & Availability:**

Risk: The reconstructions of 3D data can be distorted because of insufficient input data or input data of low quality.

Mitigation: Introduce tight data quality criteria, mix-up data material sources, and firm up data verify steps.

- **Performance Issues:**

Risk: The system does not perform adequately in terms of speed and efficiency required in real time 3D reconstruction.

Mitigation: Perform frequent testing of the performance, optimize any algorithms being used and utilize cloud computing solutions too.

- **Integration Challenges:**

Risk: Challenges in the compatibility of the 3D reconstruction system with the current systems of software and hardware.

Mitigation: have a full-fledged integration strategy, properly test compatibility, and offer a variety of API options.

- **Market Adoption:**

Risk: Resistance to usage of new technology or lack of awareness resulting in slow adoption rate.

Mitigation: Come up with a good marketing plan, offer thorough training and support and demonstrate good ROI to the potential clients.

- **Regulatory Compliance:**

Risk: Failing to comply with the changing data privacy and security laws in various jurisdictions.

Mitigation: conduct reviews and revision of compliance measures, seek legal advice and establish effective data protection procedures.

- **Resource Constraints:**

Risk: Lack of talented workforce or computing resources to cover project needs.

Mitigation: Work out a flexible resource allocation strategy, invest in training your staff and think about serious partnerships or outsourcing.

- **Scope Creep:**

Risk: Problems about deleting projects that go over budget and take too long.

Mitigation: Enforce rigorous change procedures, create well defined project boundaries, and review project scope on a regular basis against project objectives.

- **Competitive Pressure:**

Risk: Competitors advancing at fast rates can make our solution to be less competitive.

Mitigation: Carry out market research on a constant basis, ensure a dynamic development process, and emphasize on distinguishable value propositions.

- **Financial Constraints:**

Risk: Projects can be affected by unforeseen expenses or finances.

Mitigation: Keep close financial records, hold alternative funds and investigate wide-range funding mechanisms.

Such risk list shall be periodically reviewed and updated throughout the life of any project, especially when any development iteration is coming to an end. By anticipating and

managing these risks in a proactive manner we hope to increase the probability of success of the project and provide a successful 3D reconstruction solution.

Chapter 2

Requirement Specification and Analysis

The requirements and analytical requirements of this software system as required in this chapter are outlined. The aim is to deliver a concise and accurate Software Requirement Specification (SRS) which aims at filling the gap between technical development and commercial goals. Further, there will be analytical views including use case diagrams, entity-relationship diagrams, and data dictionary that will not be left out to guarantee the overall comprehension of the system functionality and design. This sequenced process will act as the basis toward the future phases of development and will help in steadying itself both with regards to technical objectives and perception model of stakeholders.

Requirement Specification

2.1. Functional Requirements

Our project, GVI Mapper, has the following functional requirements: they define what exactly our users need and expect in order to make the system reach the goal successfully.

Table 2.1: Functional Requirements

S. No.	Functional Requirement	Type	Status
1	The system shall accept multiple 2D images or video input for 3D reconstruction.	Input	Implemented
2	The system shall process and align input images to create a 3D point cloud.	Processing	Implemented
3	The system shall generate a textured 3D mesh from the point cloud data.	Output	Planned
4	The system shall allow users to annotate and label specific parts of the 3D model.	Annotation	In-Progress
5	The system shall support collaborative viewing and editing of 3D models.	Collaboration	Planned

2.2. Non-Functional Requirements

The non-functional requirements will outline the operation of the GVI Mapper project, pointing out the areas concerning its performance, usability, and general efficiency. Such requirements guarantee that the system is not only functional but should satisfy the very high requirements of user experience and efficiency in its work. A non-functional requirement table of the project is as expressed below.

Table 2.2: Non-Functional Requirement

S. No.	Non Functional Requirements	Category
1	The system should load 3D models within 5 seconds for a smooth user experience.	Performance
2	The system should be available 99.9% of the time, excluding scheduled maintenance.	Availability
3	The system should be able to handle up to 1TB of data for 3D models and associated metadata.	Scalability
4	The system should allow users to perform basic operations with minimal delay (within 1 second).	Performance
5	The system should be easy to maintain, with modular code and detailed documentation for future updates.	Maintainability

2.3. System Use Case Modeling

In this column, we will specify the different activities that are undertaken by the users and the system, which conveys the objectives of the users to attain when undertaking a system functionality. A use case is an instance of a series of steps taken by the system so as to deliver an observable outcome and which is of value to the user. With drawing the interactions, we will be able to make sure that the system will satisfy the needs of a user. As shown in the following use case diagram, the most important system functionalities are identified and the most important actors and their objectives are defined which can help to explain how the system is supposed to be used in reality.

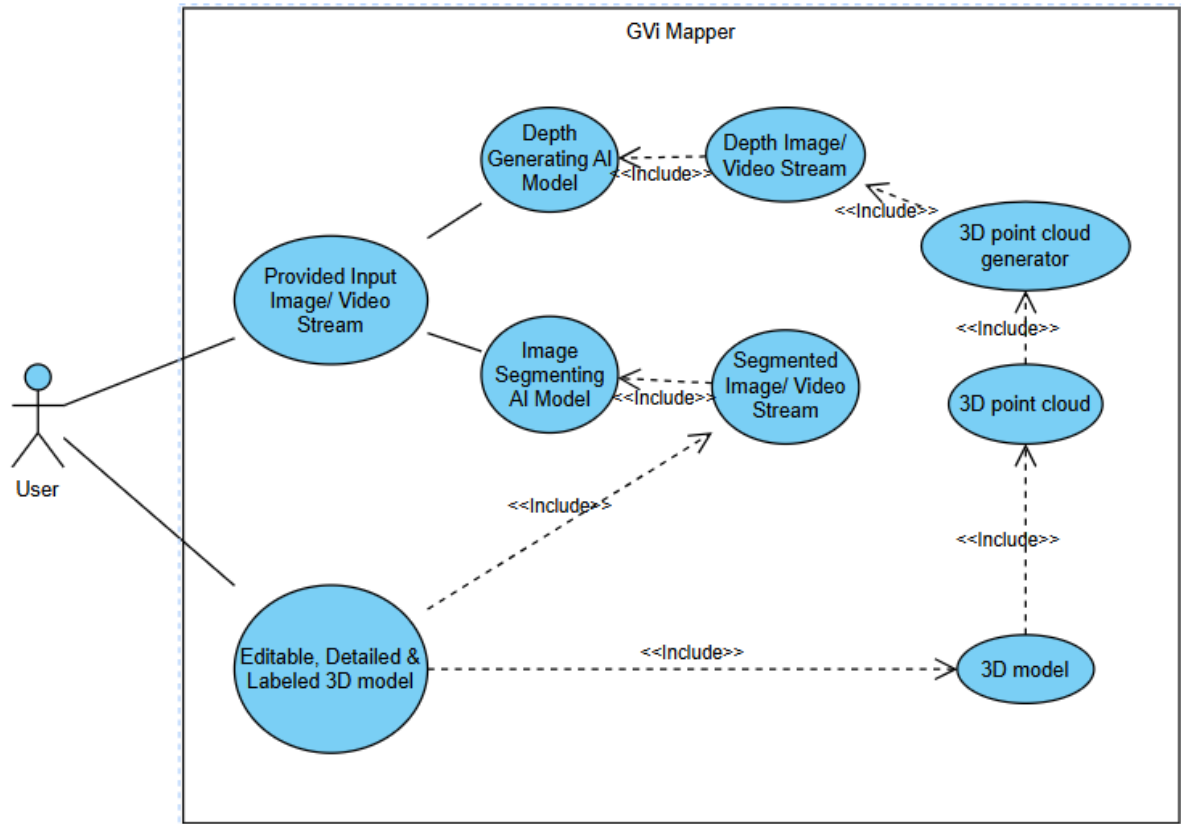


Figure 2.1: Sample Use case Diagram

2.4. Fully-dressed Use Cases

Use Case GV001:

In this use case, user interacts with the system known as GVI Mapper and enters an input image or Video Stream

Table 2.1: Use Case 1

Use Case ID:	GV001		
Use Case Name:	Provided Input Image or Video Stream		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena

Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	Users, GVI Mapper System		
Description:	The User will enter his desired input Image or Video stream into the system from his computer		
Trigger:	User enters the “Upload file” button		
Preconditions:	User is using the app or system “GVI Mapper”		
Post conditions:	After the Image or Video stream has been entered into the system. The system will pre-process it and send it to the two AI models (Image Segmentation model & Depth Generator model).		
Normal Flow:	Actor	System	
	The user uploads an Image or Video file	- The system pre-processes the Image - The system pre-processes the frames in Video - The system sends the Image or Video stream into the Depth Generator and Image Segmentation model	
Alternative Flows:	If the Image or the Frame size of the video is too small. The system will redirect the user to upload screen.		
Exceptions:	If the entered data is not an Image or Video stream. The system will reject the file and display an error message that “The file is not in a compatible format”.		

Use Case GV003:

In this use case, the AI model Depth Generator takes the Input image or frames of Video from send from the system and makes calculation and predictions of the Depth image of the Image data.

Table 2.1: Use Case 1

Use Case ID:	GV003		
Use Case Name:	Depth Image AI generator module		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena
Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	GVI Mapper System		
Description:	The module will calculate and predict the Depth of the Input Image based on the details present in the Input Image		
Trigger:	The Systems sends the entire Image or Frame data to the Depth Image Generator Module		
Preconditions:	The file has been uploaded and processed successfully into the system		
Post conditions:	After the Depth Image has been generated. It either waits for the next Image data or waits for the program to finish its execution.		
Normal Flow:	Actor	System	
	The system sends the uploaded Image or Video data to the Depth Image Generator module	- The Module performs the necessary calculation and operation from the given details of the Image - It produces the predicted Depth Image based on its calculations	
Alternative Flows:	If the Image data is noisy or is not an environmental data. It may result in faulty or terrible predictions		
Exceptions:	If the input Image data is too small to perform operations on than it will generate an error “The image data size is too small to perform the required operations”.		

Use Case GV005:

In this use case, the Depth Image Generator AI module creates the required Depth Image data which will display the necessary Depth at each region of the image which will be used in the 3D point cloud generator.

Table 2.1: Use Case 1

Use Case ID:	GV005		
Use Case Name:	Depth Image/ Video Stream		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena
Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	GVI Mapper System		
Description:	The Depth Image or Video stream is computed which will then be send to 3D point cloud generator.		
Trigger:	The completion of the Depth Map AI generator module in computing Depth Image		
Preconditions:	The Depth Map AI generator is computing the Depth Map of the Input data and then making predictions on calculating its Depth Map		
Post conditions:	After the Depth Image is generated, the generated Image data is then sent to 3D point cloud generator.		
Normal Flow:	Actor	System	
	The Depth Image Generator generates the Depth Image	The system upon its generations sends the data to 3D point cloud generator	
Alternative Flows:	If the Image or the Frame size of the video is too small. The system will redirect the user to upload screen.		
Exceptions:	N/A		

Use Case GV007:

In this use case, the 3D point cloud generator generates the 3D point cloud by taking in the Depth Image or Video stream generated from the Depth Image Generator.

Table 2.1: Use Case 1

Use Case ID:	GV007		
Use Case Name:	3D point cloud generator		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena
Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	GVI Mapper System		
Description:	The 3D point cloud generator generates the 3D point cloud using the Depth Image or Videos stream		
Trigger:	The system sends the Depth Image or Video stream data to the 3D point cloud generator		
Preconditions:	The system has sent the Depth Image data to the 3D point cloud generator		
Post conditions:	The 3D point cloud model has been generated using the 3D point cloud generator.		
Normal Flow:	Actor		System
	The system sends the Depth Image data to the 3D point cloud generator		The 3D point cloud generator computer computes all the necessary calculations for the models generation and generates it
Alternative Flows:	If the Depth data is noisy then a noisy 3D point cloud model will be generated		
Exceptions:	N/A		

Use Case GV008:

In this use case, the 3D point cloud generator has generated a 3D point cloud model which will be further processed before it reaches its final stage.

Table 2.1: Use Case 1

Use Case ID:	GV008		
Use Case Name:	3D point cloud generator		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena
Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	GVI Mapper System		
Description:	The 3D point cloud generator has generator has generated a 3D point cloud model which will be then send to be further processed		
Trigger:	The 3D point cloud generator has generated the 3D point cloud model		
Preconditions:	The 3D point cloud generator has generated the 3D point cloud model		
Post conditions:	The 3D point cloud model is further processed by removing the noise in the data		
Normal Flow:	Actor	System	
	The 3D point cloud generator generates a 3D point cloud model	The system pre-processes the 3D point cloud by removing noise before finalizing	
Alternative Flows:	If the generated 3D point cloud model has too much noise then it will be difficult to produce an accurate 3D point cloud model.		
Exceptions:	N/A		

Use Case GV009:

In this use case, the system converts the given model to the users desired type of model which is a point cloud model or a mesh.

Table 2.1: Use Case 1

Use Case ID:	GV009		
Use Case Name:	3D Model		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena
Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	User, GVI Mapper System		
Description:	This is the final stage of 3D point cloud where the 3D point cloud is converted to mesh or finalized as point cloud as a result		
Trigger:	The 3D point cloud model finishes being pre-processed		
Preconditions:	The 3D point cloud model has finished being pre-processed		
Post conditions:	Upon the user choice, the final 3D model will be generated according to the users desired type		
Normal Flow:	Actor	System	
	The user selects his desired 3D model type	- The system converts the desired 3D model according to the users desired type - The 3D model is finalized	
Alternative Flows:	If the system fails to generate a 3D mesh, then system will only finalize the point cloud as the final model		
Exceptions:	N/A		

Use Case GV004:

In this use case, the system sends the Input data to Image Segmentation module to convert the given image to a segmented Image data.

Table 2.1: Use Case 1

Use Case ID:	GV004		
Use Case Name:	Image Segmentation AI module		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena
Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	GVI Mapper System		
Description:	The Image segmentation module takes the Image data and performs calculations and operations and generates the segmented Image data		
Trigger:	The system sends the Image data to the Image segmentation module		
Preconditions:	The system has taken the input data from the user and pre-processed it		
Post conditions:	The Image segmentation AI module has generated a Segmented Image		
Normal Flow:	Actor		System
	The system has sent the Image data to the Image Segmentation AI module		- The Image Segmentation Module generates the segmented Image - It Labels each and every pixel according to their labels
Alternative Flows:	If the Image or the Frame size of the video is too small. The system will redirect the user to upload screen.		
Exceptions:	If the entered data is not an Image or Video stream. The system will reject the file and display an error message that “The file is not in a compatible format”.		

Use Case GV006:

In this use case, the Segmented Image has been generated by Image Segmentation module which would be later sent to create an interactive and editable 3D model

Table 2.1: Use Case 1

Use Case ID:	GV006		
Use Case Name:	Segmented Image/ Video stream		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena
Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	GVI Mapper System		
Description:	The Segmented Image is generated through Image Segmentation Module		
Trigger:	The completion of the generation of the Segmented Image through Image Segmentation Module		
Preconditions:	The completion of the generation and labeling process of the segmentation module		
Post conditions:	After the Segmented data is generated, it will be sent to the last stage where an editable and interactive 3D model is generated		
Normal Flow:	Actor		System
	The Image Segmentation module generates the Segmented Image		The system sends the Segmented Image data to the final stage
Alternative Flows:	If the Image is noisy then terrible segmented data is generated		
Exceptions:	N/A		

Use Case GV002:

In this use case, the segmented data and 3D model is used to create an editable and interactive 3D model which may help users in getting the desired information from the model with the detailed 3D data

Table 2.1: Use Case 1

Use Case ID:	GV002		
Use Case Name:	Editable, Interactive and detailed 3D model		
Created By:	Abdus Sami Chheena	Last Updated By:	Abdus Sami Chheena
Date Created:	19-12-2024	Last Revision Date:	19-12-2024
Actors:	User, GVI Mapper System		
Description:	This is the last stage, where a 3D model as a base and segmentation data is used to create an interactive and editable 3D model to give the user their desired form of information		
Trigger:	The system sends both segmented Image data and 3D point cloud data		
Preconditions:	The segmented Image and 3D point cloud data has been generated		
Post conditions:	After the detailed 3D model is generated, the user can use it however to filter out any undesirable information to create a new 3D model or use the previous information		
Normal Flow:	Actor	System	
	The User can engage with the 3D model based on the options provided to get his desired information	The system sends the information to the user	
Alternative Flows:	N/A		
Exceptions:	N/A		

2.5. Domain Model

The GVI Mapper project domain model describes the main entities and their interactions within the application and gives a clear picture of the main components of the system that is used in creation and development of 3D models and point clouds. It is representative of the

interactions between users, input data, AI models, and the outputs, which can be used as a basis of design and implementation of the project. The next sections specify the primary entities and their relations, which provides an idea about the architecture of the system.

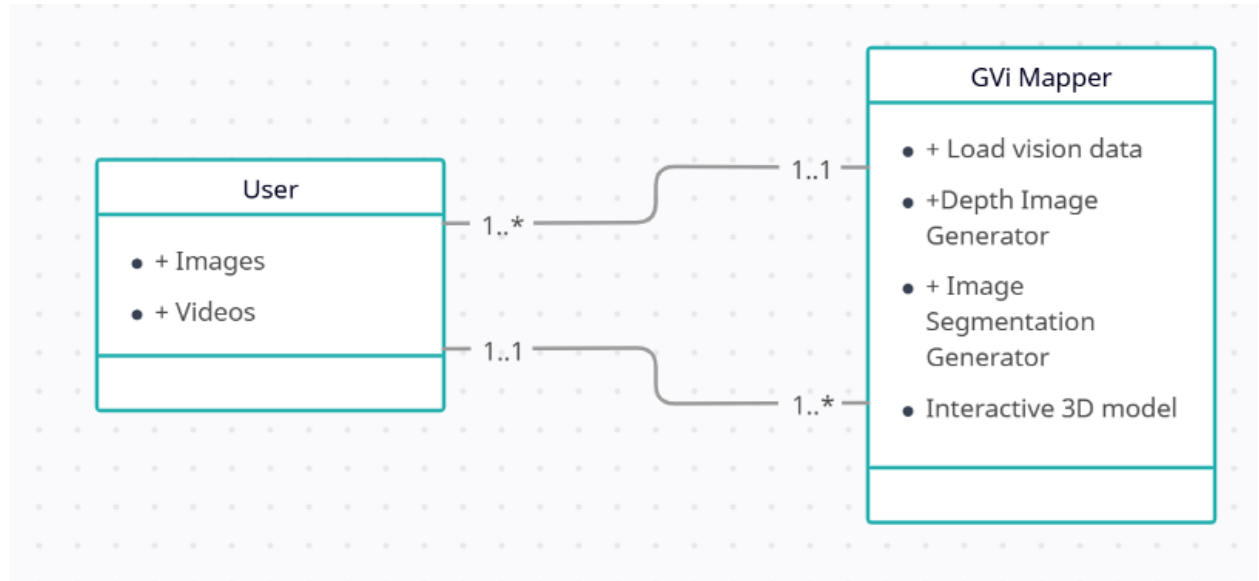


Figure 2.3: Domain Model

Chapter 3

System Design

The chapter describes the architectural and design methodology as the system, providing a blue print that allows consolidating expectations between the requirements and the implementation. It concentrates on the structure and behavior of the software such that, the project presents both functional and non functional requirements. The system design of this 3D reconstruction project places focus on the design being modular, can scale and be maintained so that it is the best performing and is usable. The most important ones are recognizing the key pieces of software, describing how they interact as well as which design patterns should be applied in case certain issues associated with the scope of the project appear. The sections below elaborate the design, behavior and structure of the system and the principals and constraints upon which decisions were made.

3.1. System Architecture

The architecture of our 3D reconstruction system has been made robust and large scale capable of achieving a performance both efficient and accurate in dealing with complex data in scenes and coming up with an accurate 3D representation. The system is made up of some of its important components that are coordinated to produce quality results.

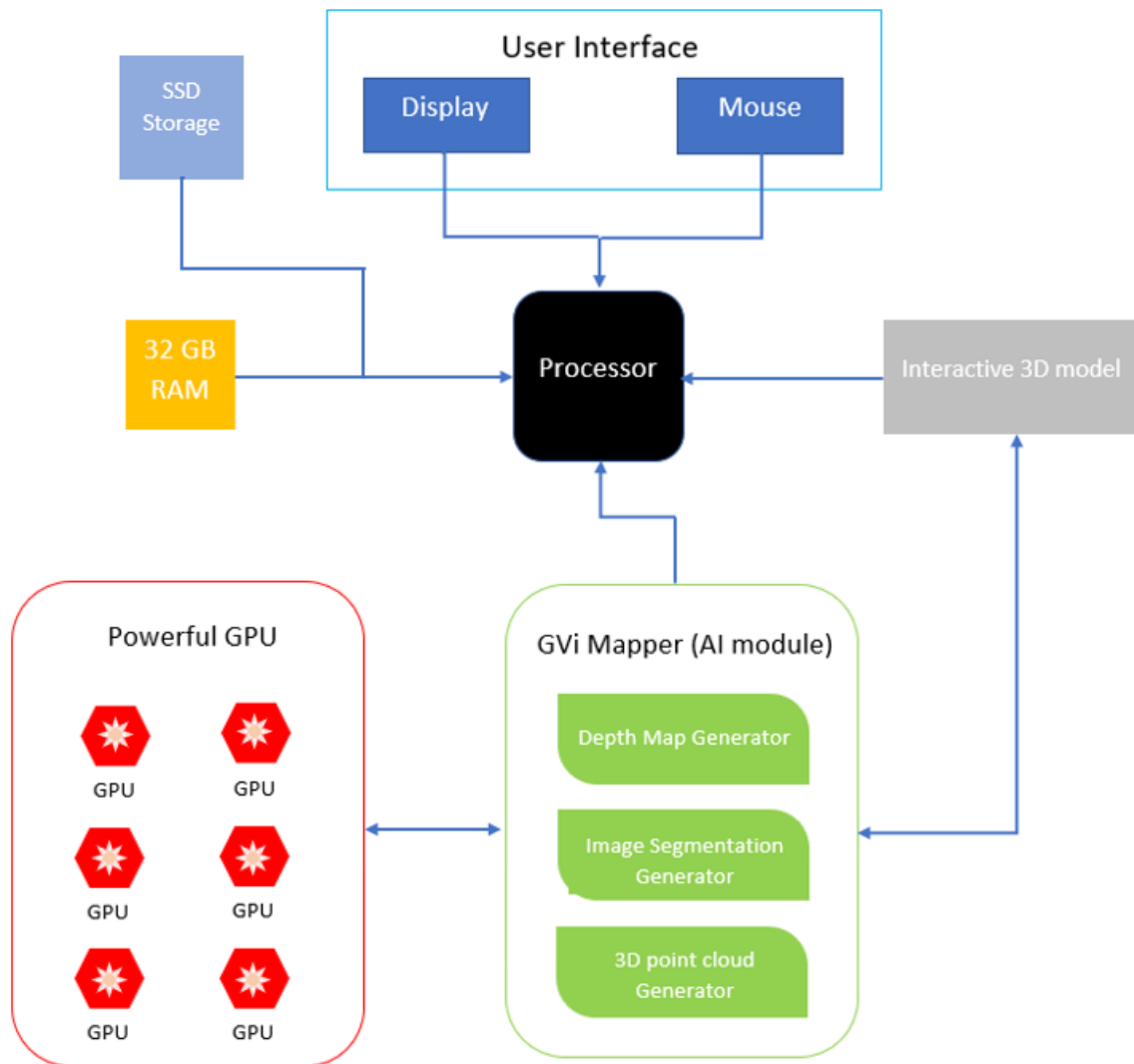
Hardware Components

- **Processors:** Parallel GPU image processing and reconstruction algorithms with high-performance GPUs.
- **Memory:** Minimum 32GB RAM to handle large datasets and complex computations.
- **Storage:** Fast SSD storage for quick data access and model storage.

Software Components

- **Programming Languages:** Python is the core language.
- **Frameworks:** PyTorch for machine learning operations
- **Libraries:** OpenCV for Image processing, Numpy for Data Manipulation and Open3D for processing 3D data.

- **IDEs:** Visual interaction along with the GPU access and collaboration of multi-purpose assistance with visual studio.



3.2. Workflow Design

The design of the workflow in the 3D reconstruction system includes a few main steps so that to guarantee the creation and implementation of a perfect AI model.

- **Data Acquisition and Preprocessing:**
- **Data Collection:** Collect input Hypersim, Kitty, VKitti2 data thereby providing as diverse input data as well as saturative input both indoor and outdoor data

- **Preprocessing:** Improvement of data quality involves completing tasks e.g. reduction of noise, normalization, and augmentation before passing to the training process.
- **Model Training and Evaluation:**
- **Training:** Use the preprocessed data to develop the AI models including the depth generation and segmentation networks to optimize them to interpret and construct the 3D structures in an accurate manner.
- **Evaluation:** Measure the performance of the model based on the right metric, like accuracy, precision and recall, so that the models that are produced are up to the mark.
- **Deployment:**
- **Integration:** Train the models and put them through the 3D reconstruction pipeline, so that the system will be able to work on new input data and create 3D models in real-time.
- **Production Environment:** Move the integrated system into the production environment, make it run smoother and more effective to the people who would use it.

This is a step-by-step workflow which will systematically develop the 3D reconstruction system so as to ensure every other phase being not only executed, but also neatly done.

3.3. UML diagrams

UML diagrams are essential in the development of the structure and behavior of the system and the context of GVI Mapper. These diagrams give a clear insight on how components of the 3D reconstruction system connect with each other and the users. The interaction between the user facilitated by the use case diagrams, the structure of the system visualized by the use of class diagrams, and the sequence of the operations graphically explained on sequence diagrams make the design understandable and customisable. These visual tools make sure that architecture of the system corresponds to its goals, are easy to communicate so that they can be implemented effectively.

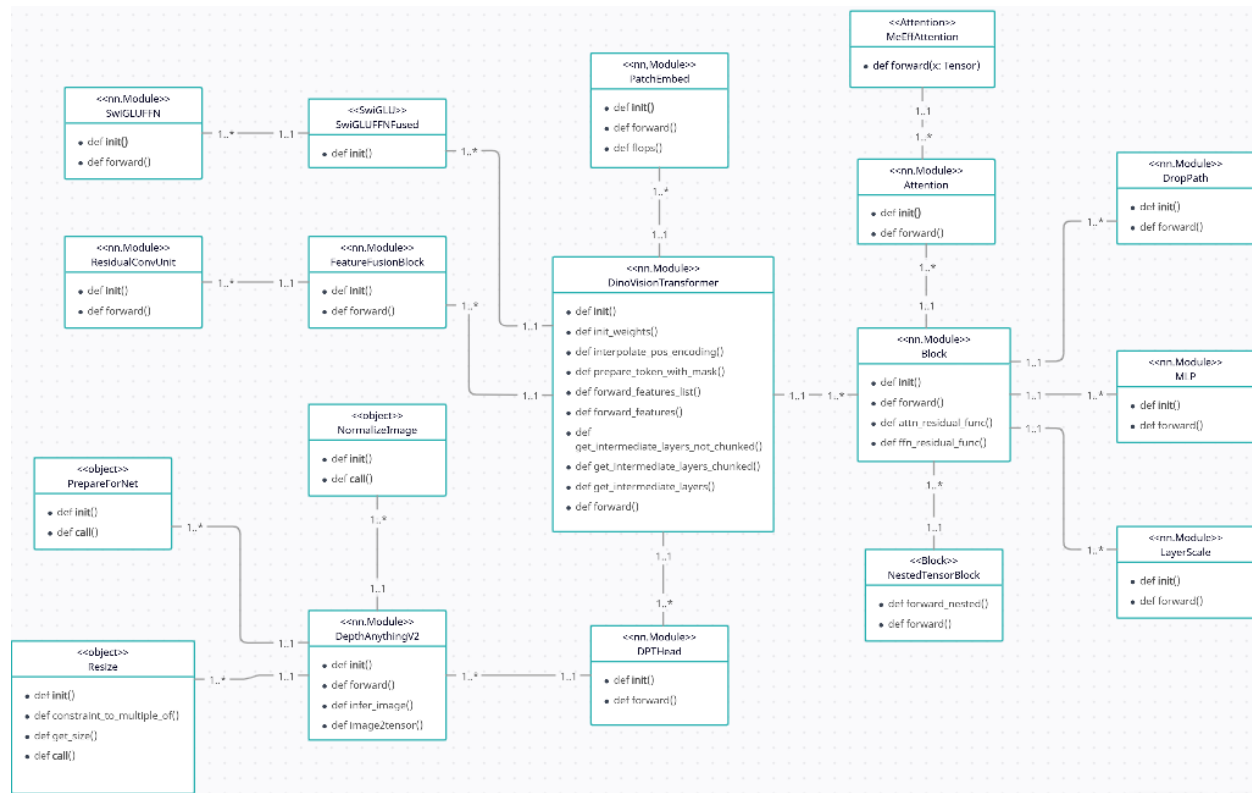


Figure 3.1: Class Diagram

When combined with class diagrams, the sequence diagrams give the complete idea about how the system is to behave because it explains how different objects interact with each other at specific processes. Sequence diagrams, as shown in Figure below, can be used within the context of the system under 3D reconstruction, which describes the communication of the components of the system (including the user interface, processing modules, and the AI model) during such tasks as the 3D model generation or export. These diagrams are helpful in showing how the operations are performed and how flow of data occurs as complementary to the relations and dependency in the class diagram. Collectively, these diagrams provide an effective and descriptive picture of the structure of the system and variability of interactions.

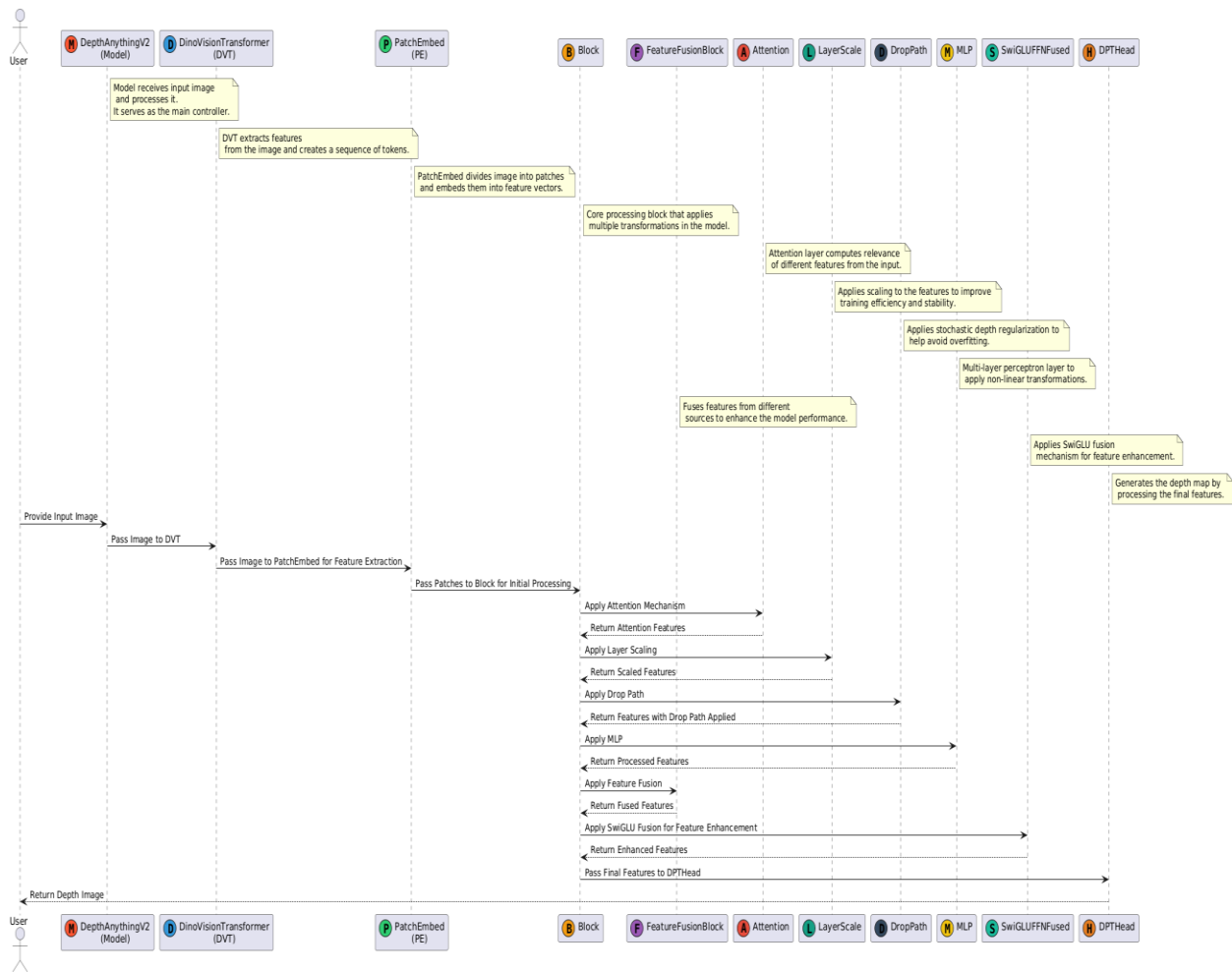


Figure 3.3: Sequence Diagram

3.4. Dataflow and preprocessing pipelines

A thorough dataflow and preprocessing pipeline is used in our 3D reconstruction project so that high quality input can be given to reconstruction algorithms. The pipeline has a number of important steps:

1. Data Acquisition

- For Indoor scenes, we have used Hypersim dataset for their Indoor scenes
- For Outdoor scenes, we have used Kitti and Vkiti2 dataset for their Outdoor scenes

2. Data Ingestion

- GitHub will be used to load the Hypersim Dataset
- The Kitti and Vkitti2 dataset will be downloaded on their own page

3. Preprocessing

- Normalize and standardize input data
- Apply image enhancement techniques (e.g., contrast adjustment, denoising)
- Dividing the images into Patches and embedding them to extract features

4. Data Visualization

- Apply 3D point cloud to editing and representation of 3D data

3.5. Activity Diagram

Activity diagram It is a graphical representation of the flow of activities or steps of accomplishing a specific use case. They give large picture view of the interaction between various components and actors to accomplish certain goals in the system. In the 3D reconstruction project, activity diagrams have been constructed to guide through the step by step process concerning tasks like acquisition of the images, generation of 3D models, and exportation of the models to guarantee that the flow of operations is understood. Such diagrams also assist in pointing out possible bottlenecks and improving the level of efficiency of the design and implementation of the system.

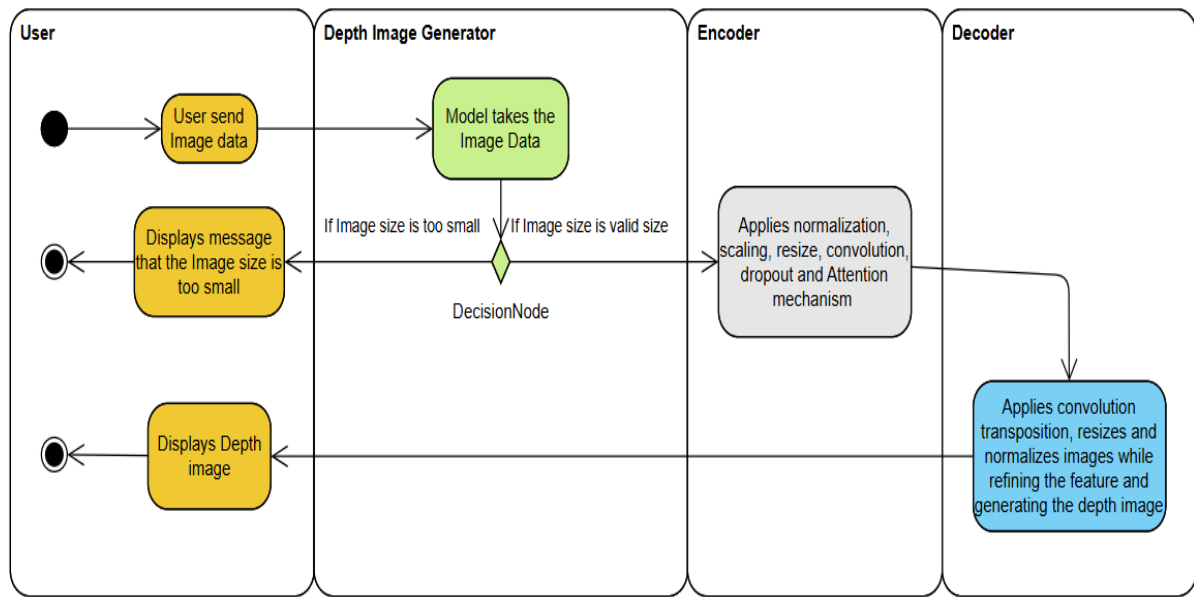


Figure 3.2: Activity Diagram

3.6. Algorithms and model design

The algorithms and model design component is the core part of the AI system where procedures and framework are laid out on how the system functions. In the case of GVi Mapper project, this means developing an appropriate model architecture most suited to the problem area, specifying the layers and the components and describe the training process of the model. This section describes how the selected algorithms made sense, what optimisation techniques have been adopted, and how the model architecture fits the project goals. This section gives an overview of the complete coverage that has been done so as to make the system accurate and efficient.

In our project we have decided to use Depth Anything V2 architecture that consists of following important components.

Model Architecture

1. Input Layer:

- **Function:** Individual images are processed in terms of resizing and cropping, which ensures uniformity in the data set.
- **Configuration:** The images are resized to have the shortest side as 518 pixels and are then picked at random crop of dimension 518 x 518.

2. DINOv2 Backbone:

- **Function:** Can be used as the feature extractor, able to obtain high-level representations on input images.
- **Configuration:** Utilizes the DINOv2-G (giant) encoder for the teacher model, and DINOv2 small, base, or large encoders for student models.

3. DPT Decoder:

- **Function:** Interprets features from the backbone to predict depth information, enabling dense depth estimation.
- **Configuration:** Incorporates transformer-based mechanisms to handle complex spatial relationships.

4. Output Layer:

- **Function:** Generates the final depth map, providing pixel-wise depth predictions for the input image.
- **Configuration:** Outputs a depth map corresponding to the input image dimensions.

Training Process

The training methodology is structured into distinct phases:

1. Teacher Model Training:

- **Data:** Approximately 595,000 synthetic labeled images are utilized to train the teacher model, ensuring high-quality depth supervision.
- **Optimization:** The Adam optimizer is employed with a learning rate of $5e-6$ for the encoder and $5e-5$ for the decoder.
- **Iterations:** Training is conducted over 160,000 iterations with a batch size of 64.

2. Pseudo-Label Generation:

- **Process:** The trained teacher model generates pseudo-depth labels for over 62 million real unlabeled images, bridging the gap between synthetic and real-world data.

3. Student Model Training:

- **Data:** Student models are trained on the pseudo-labeled real images to enhance generalization capabilities.
- **Optimization:** Similar optimization parameters are applied, with adjustments based on model scale (small, base, large).
- **Augmentation:** Horizontal flipping, strong color distortions and CutMix techniques are used as robustness enhancement techniques.

3.7. Scalability and performance considerations

With AI systems such as the GVI Mapper project increasingly growing in size and complexity, ensuring scalability and performance is becoming increasingly important so that efficiency and responsiveness can be retained. This portion will go into how to handle the new data loads and computational requirements, which should make the system not only reliable but also flexible.

Optimizing Model Training and Inference Speed

Increasing the training and inference speed of models is crucial to real-time applications and user enjoyment. Important optimization methods are:

- **Hardware Acceleration:** Calculations can be auto-accelerated by taking advantage of specialized GPUs or TPUs, or Graphics Processing Units and Tensor Processing Units, respectively. These accelerators deal with parallelism operations in an efficient way and can lessen training time and enhance inference time.
- **Parallel Processing:** Sharing of work over a number of processors or computer, enables several computer programs to work at the same time, thereby boosting performance. Parallel processing of data and training of the model has the potentiality of saving turnaround time, particularly in the large datasets scenario.
- **Model Optimization Techniques:** Model pruning, quantization to optimize the model and knowledge distillation to minimize its size and also optimise its complexity may be used to optimize the model and shrink the size and complexity of the model without compromising on the accuracy. The methods increase the rate at which inferences are obtained and reduce the amount of resources being used.

Deployment Strategies: Cloud-Based Solutions and Edge Computing

When to choose one deployment technique over the other, performance, scalability and resource utilization becomes vital in this case. There are two major ways:

- **Cloud-Based Solutions:** Models deployed on the cloud platforms are scalable and flexible. Cloud services offer a strong infrastructure with the capacity to undertake massive computational workloads and huge data sets. They ensure a consistent performance amidst different levels of demand due to the allocation of resources as and when they are required, enabling the scalability.
- **Edge Computing:** Edge computing entails models on edge devices where AI models are running locally on edge devices like smart phones, embedded systems or IoT devices. The strategy limits latency and bandwidth consumption since the data are processed closer to the source, and it benefits the real-time applications, as well as the situations with limited connectivity.

Considerations for Scalability

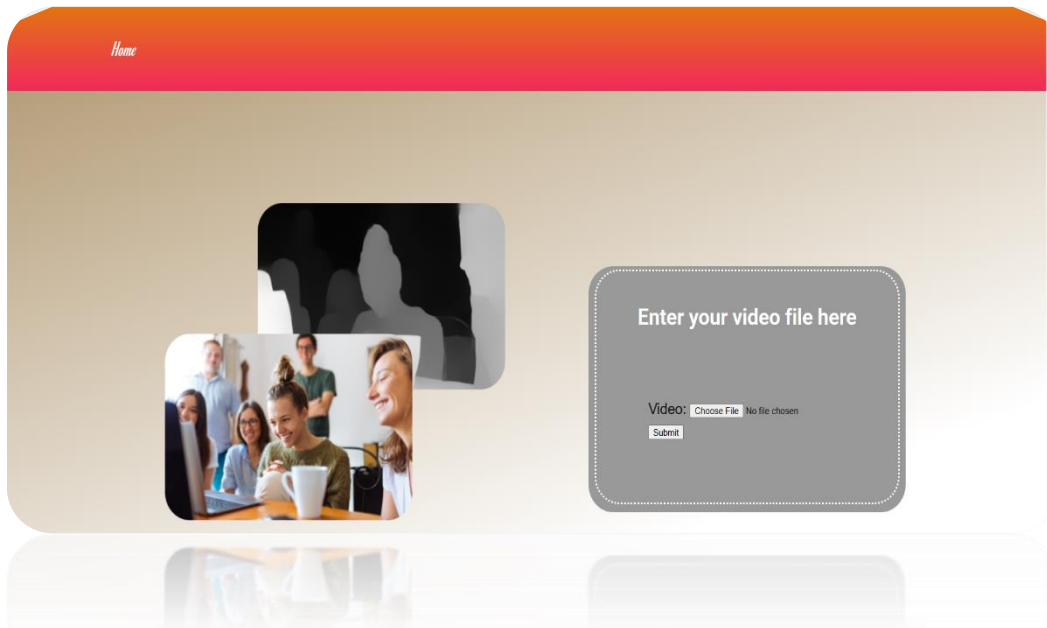
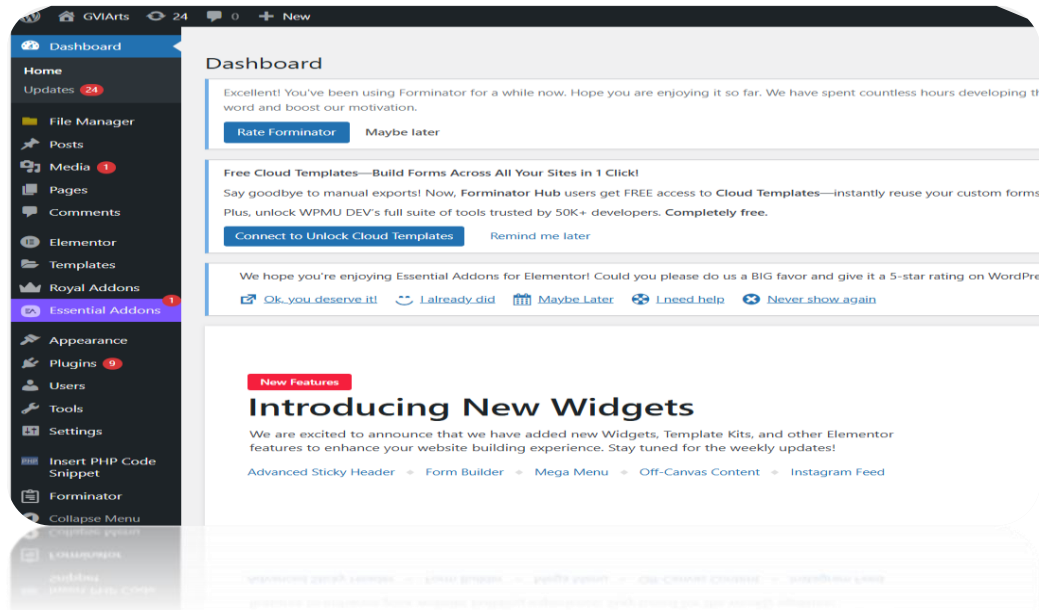
So that the system will be able to support any future growth:

- **Modular Architecture:** Such a modular design of the system is that the various elements can be scaled individually according to demand. This flexibility makes it efficient in the use of resources and makes it easy to maintain.
- **Resource Management:** The resource management strategies like dynamic resource allocation and load balancing should be implemented to ensure the best performance considering different work loads. Dynamics causes can be avoided by observing and balancing resources where bottlenecks and system responsiveness can be achieved.

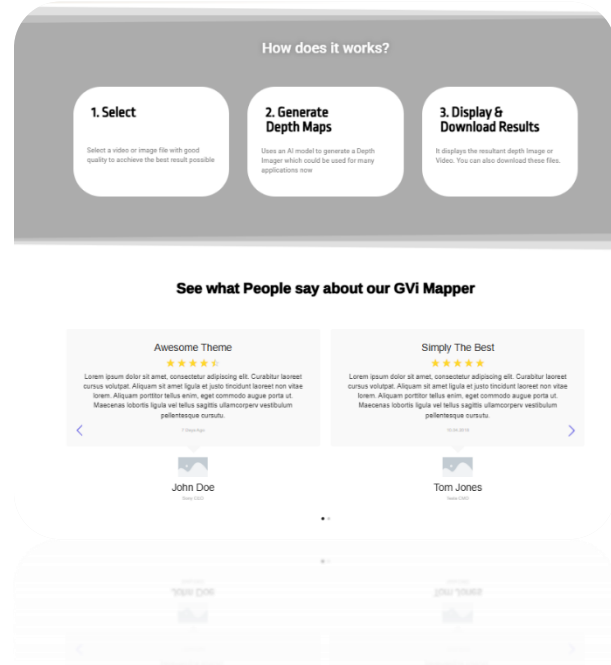
Combining these optimization solutions and deployment approaches, the GVi Mapper project will have a scalable, highly effective AI system that can overcome and meet growing needs and provide efficient and real-time 3D mapping solutions.

3.8. User Interface Design

For our user interface, we have developed and published a website which will take all the users video data and use it to generate a depth video. We used WordPress for our website designing software so that we could develop an interactive and good-outlook website. We used pantheon sites free WordPress dashboard for developing our frontend website using Elementor widgets and deployed it on the internet.

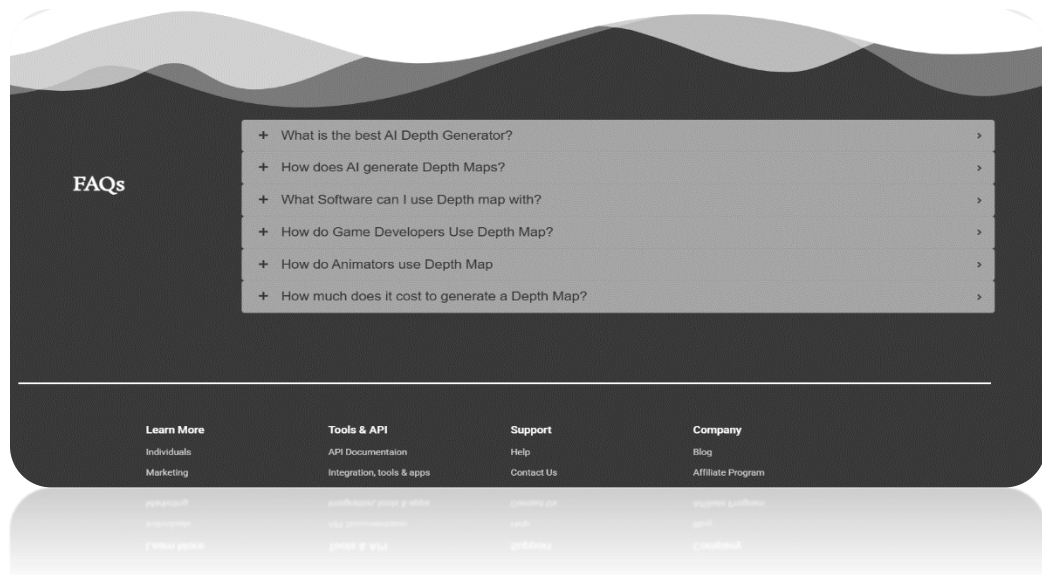


The website has been designed to be interactive and engaging for the user. We have also provided our users with instruction on how to use our software. You can verify the value of our software by reading the reviews of our customers below the instruction for the credibility of our software.

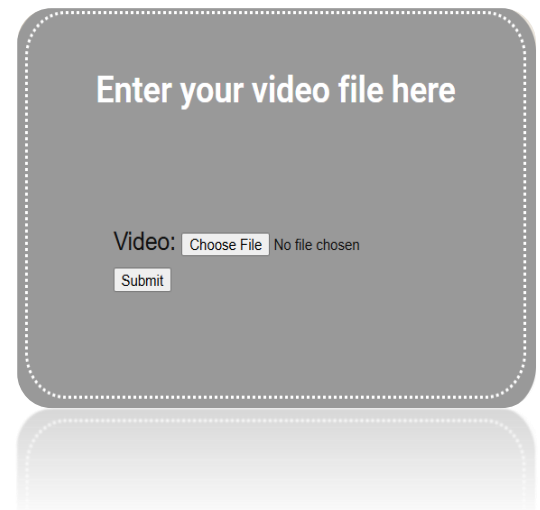


We have also shown our users of the widespread examples of use of our application in various industry for our users understanding of the value of our software for advertisement purposes.

In the end to entertain any FAQs, company and details of our application use, software and contact for additional verification and details to our clients or customers. We have prepared a footer for our users help.



For our users video upload function, we have designed a PHP form using WordPress PHP snippet plugin which takes users video and direct it WordPress dashboard media library. The user can enter the video here for user desires the software to perform its functions on. When the video is uploaded, our code the WordPress theme function creates a short code for one video uploaded by user and one uploaded by GVi Mappers backend. When the submission button is pressed it leads the user to our display video page to demonstrate results



The screenshot shows a video upload form with a dashed border. The title 'Enter your video file here' is at the top. Below it, the label 'Video:' is followed by a 'Choose File' button and the text 'No file chosen'. At the bottom, there is a 'Submit' button.

Once the video is already uploaded, our backend processes it through our AI model to create both a depth and a color depth video these are further forwarded to WordPress media library with specific names. Through the unique Philosopher of Heated Periods code in WordPress guideline functioning, those files are read by the code and also are interfaced with the name the original video files with the creation of a short code that can in turn be presented on the video widget of the Display video web page in PHP. It shows all the three videos namely original video, depth video and colored depth video.



Chapter 4

System Development

This coverage highlights the actuality of the GVI Mapper project with the project execution shifting to inter modal mode. It points out how the system architecture and processes have been converted into running code that corresponded to the design requirements. The roll out process usually involves unexpected obstacles that may arise, e.g., passing through the intricacy of the current software, compatibility problems, or dealing with the restrictions of present tools and materials. In this case, we record the plans and the means by which these challenges were tackled and the repeated effort towards the eventual creation of the system. Also, in this section we will have a chance to demonstrate the technical sophistication and the effort that went into bringing a solution that is stable, efficient, and can be scaled.

4.1. Coding Standards

This part explores the actual implementation of the GVI Mapper project moving it beyond conceptual design into the actual application form. It elucidates the translation of the architecture and workflows of the system and the mechanism that has been converted into working code that was ensured to go in line with the design specification. The implementation step may result in certain unseen obstacles like the complexity of the existing software, compatibility problem, and limitations of existing tools and resources. In this case, we trace down the tactics and solutions that we have used to overcome these challenges and how the back-and-forth process eventually culminated to successful creation of the system. On top of that, this section gives a chance to demonstrate the technical depth and the effort made to build a full-featured scalable and usable solution.

Indentation:

It conventional indentation with 4 spaces per indentation level is in line with PEP 8 style guide of Python. The indentation is always consistent in all the begin blocks such as classes, functions and loops as well as conditional block. Every indent is performed with spaces and no tabs are utilized.

Declaration:

Classes are declared following the standards of the CamelCase types of declaration and the declaration of the functions or methods follow the standards of snake_case declaration. Even the variable declaration is also as per the snake case standards.

Naming Conventions:

The variables names have been selected to show specific name of each component to the model and its functionality. The names of variables were aligned to the intent of the layers of the model, related projection, preprocessing operations as well as to the logic of given conditions in statements. Such a method makes sense clearer and one can easily define the flow of data and the services carried out at each point of the model.

Statement Standards:

A new line was made after each statement and it was terminated appropriately to make them more comprehensible. Multi-lines statements were typed in with proper breaks and alignments. Comments were also applied reasonably within the scope of expressing complicated logic where a comment on a specific code line was referred to as inline and longer lines of code were referred to as block comment. This practice is in line with the best practices in code quality of AI projects.

Following these coding practices, we guaranteed that the codebase of the GVi Mapper project is organized conveniently, which favors the subsequent maintenance and scalability of our project. It is very important in the field of artificial intelligence development where sophisticated algorithms and data structures need articulate and unified coding styles.

4.2. Development Environment

For this project, I used **Google Collab** and **Visual Studio** as development environments. Collab provides a many python libraries and GPU support. It helps in verifying the issues and bugs in code Along with packages integration supports which allows user to use any package without any issue. While Collab has all the packages integration support, it fails in providing enough GPU resources and a window to visualize and interact with 3D models. Visual Studio is used for such purposes for GPU support from local system and for displaying a window for visualizing 3D model.

Packages and Libraries Used:

The following packages and libraries were employed during the development of the project:

- **PyTorch:** Used for specific tasks where certain operations or model components were more efficiently implemented in PyTorch. PyTorch's dynamic computational graph made certain parts of the model easier to work with.

- **OpenCV:** Employed for image preprocessing and manipulation, including resizing, normalization, and converting images to tensor formats.
- **Torchvision:** A package used for visualizing the graph of architecture of the model.
- **TorchInfo:** A package used for visualizing the details of the models and displaying them the complete architecture details.
- **NumPy:** Used extensively for array manipulation and mathematical operations on the dataset and intermediate model outputs.

Third-Party Libraries:

- **DinoV2:** A third-party pre-trained vision transformer model that was integrated into the project for feature extraction and image processing.
- **Open3D:** Although not utilized in the core model, this library was considered for potential point cloud and 3D model operations as part of future extensions to the project.

The choice of these tools and libraries was based on the need to provide the project with the latest technologies and solutions that are already implemented so that the development of the software could be faster, the models more precise, and the deploys efficient.

4.2.1. Hardware Requirements:

Minimum Hardware Requirements:

The project is dependent on the following minimal hardware requirements in an effort to perform and generate the best results and dependability. These are the requirements of this AI model functioning and making the work of the system go seamlessly.

- **CPU:** Intel Core i9-13900K or equivalent
- **RAM:** 24GB RAM or more
- **Storage:** 512GB SSD or HDD
- **GPU:** NVIDIA RTX 3090 or equivalent

Cloud-Based Solutions:

To perform work of higher stakes or to evade the burden of local hardware configuration, you can resort to cloud-based solutions:

- **Google Colab:** An open-source cloud computing system giving access to GPUs and TPUs, which contributes to an ideal platform in terms of machine learning and deep learning activities.

4.2.2. Software Requirements:

Necessary Software Tools & Libraries:

- **Python:** A multipurpose programming language required in data science and machine learning.
 - **Version:** Python 3.10 or upper.
 - **Installation:** Python and the dependencies may be installed with a package manager such as pip.
- **OpenCV:** A computer vision library written in C++ and distributed as Open Source, designed to be used in image and video processing application.
 - **Version:** The newest stable branch (e.g. OpenCV 4.8.x).
 - **Installation steps:** Then install OpenCV via pip.
- **PyTorch:** Dynamic and effective deep learning interface.
 - **Version:** Recent steady release (e.g. PyTorch 2.x).
 - **Installation:** Use pip to install PyTorch (ensure CUDA compatibility if GPU is used).
- **Numpy:** A fundamental library for numerical computations in Python.
 - **Version:** Latest stable version (e.g., NumPy 1.26.x).
 - **Installation:** Use pip to install NumPy.
- **Open3D:** A library designed for 3D data processing and visualization.
 - **Version:** Latest stable version (e.g., Open3D 0.18.x).
 - **Installation:** Use pip to install Open3D.
- **TorchInfo:** A library to summarize PyTorch models, providing layer-by-layer details.
 - **Version:** Latest stable version (e.g., TorchInfo 1.8.x).
 - **Installation:** Use pip to install TorchInfo.
- **TorchVision:** A library for image-based tasks, supporting pre-trained models and image processing utilities.

- **Version:** Latest stable version (e.g., TorchVision 0.16.x).
- **Installation:** Use pip to install TorchVision (ensure compatibility with PyTorch version).

4.3. Dataset Acquisition and Preprocessing

4.3.1. Dataset Selection:

For our GVI mapper project, the dataset required must hold Depth Maps or Depth Images as these are the data upon which the models will be trained on. These datasets are following

- **Kitti:**

- **Relevance:** The KITTI dataset is pivotal for 3D reconstruction and monocular depth estimation tasks, offering real-world driving scenarios that aid in developing and evaluating models like GVI Mapper.
- **Size:** The data consisted of more than 93,000 image pairs of stereo images, so there is a significant number of data to use in training and testing.
- **Quality:** KITTI images and depth are made by high-resolution cameras and accurate LiDAR sensors that guarantee the high quality of images and depth data, which are critical to the proper model development.
- **Variety:** It covers a wide range of backgrounds, such as urban scenes, rural scenes, and highway scenes where the range of lights and weather is wide, which improves the protection and generalizations of the 3D reconstructed models.

- **VKitti2:**

- **Relevance:** VKITTI2 is a synthetic alternative to KITTI which offers photorealistic virtual worlds which are extremely valuable to train models in monocular depth estimation and depth cameras including GVI Mapper.
- **Size:** The range of data includes about 43,000 RGB-D pairs of images, which also allows extensive training of models.
- **Quality:** Being a synthetic dataset, VKITTI2 allows generating ideally labeled data and dense depth maps, making it of high quality as a source of input in the development of models.
- **Variety:** It has different weather conditions (e.g., rain, fog) and lighting situations, which enables models to learn and adapt to diverse environmental characteristics and this enhances the overall performance when applied in the real-world systems.

- **Hypersim:**

- **Relevance:** Hypersim is of great importance to reconstruction and monocular depth estimation tasks relating to 3D reconstruction and synthetic interior scenes and covers per-pixel labeling as well as ground truth geometry providing photorealistic scenes.
- **Size:** The data size consist of around 77400 images that are found on 461 different scenes indoors and this makes it a reasonable amount of data to train and test models such as GVI Mapper.
- **Quality:** Since it is a synthetic data, Hypersim also shows high quality of images without noise and depth annotations, which helps to train accurate and stable 3D reconstruction models.
- **Variety:** The data has a wide variety of indoor settings with different lighting conditions and high-detailed semantic labels of each pixel which enables the model to generalize better in different indoor settings.

- **Advio Dataset:**

- **Relevance:** ADVIO is useful in visual-inertial odometry (VIO) operations and its SLAM. It gives real-life information that is gathered with smartphones, so it can be used to test the quality of VIO systems during actual operations.
- **Size:** It comprises 23 sequences which were captured in various indoor and outdoor positions. Synchronized video, IMU data (accelerometer and gyroscope) and the ground-truth 6-DoF trajectories are provided for each sequence.
- **Quality:** The data provides good inertial ground truth of the inertial navigation techniques without site of GPS or external monitoring. This renders it to be trusted even in locations that are beyond the reach of the GPS.
- **Variety:** a wide range of scenarios is included in the scope of ADVIO, such as offices, shopping malls, metro stations, and city streets. It has varied movements such as walking, stairs, escalators, and lifts with different lighting and environment, so the models are used to work in various environments.

ADVIO uses an iPhone 6s to collect synchronized monocular RGB video (60 FPS) and IMU data (accelerometer and gyroscope at 100 Hz). Ground truth is generated using pure inertial navigation with manual loop-closures, providing accurate 6-DoF trajectories without relying on GPS or external systems.

4.3.2. Data Cleaning and Preprocessing:

These datasets are mostly clean but still require some additional cleaning and pre-processing steps. These are

- **Handling Missing Values:**
 - **Deletion:** Remove rows or columns containing missing values, especially if they are minimal in number.
 - **Imputation:** Fill missing values using statistical measures (e.g., mean, median) or predictive models to maintain data integrity.
- **Outlier Detection and Handling:**
 - **Detection:** Use statistical methods (e.g., box plots, Z-scores) to identify outliers that may distort the model's performance.
 - **Handling:** Trimming removes identified outliers, capping replaces extreme outliers with predefined threshold values, and Winsorization adjusts outliers to the nearest acceptable percentile values.
- **Data Augmentation:**
 - **Purpose:** Enhance the diversity of training data and improve the model's generalization capabilities.
 - **Techniques:** Apply random rotations, scaling, translations, and other geometric transformations to simulate varied real-world scenarios.
- **Feature Scaling:**
 - **Normalization:** Scale input features to a range (e.g., 0 to 1) for uniformity, especially for image intensity values.
 - **Standardization:** Adjust features and apply standard deviation to them according to ImageNet standards

4.3.3. Data Splitting:

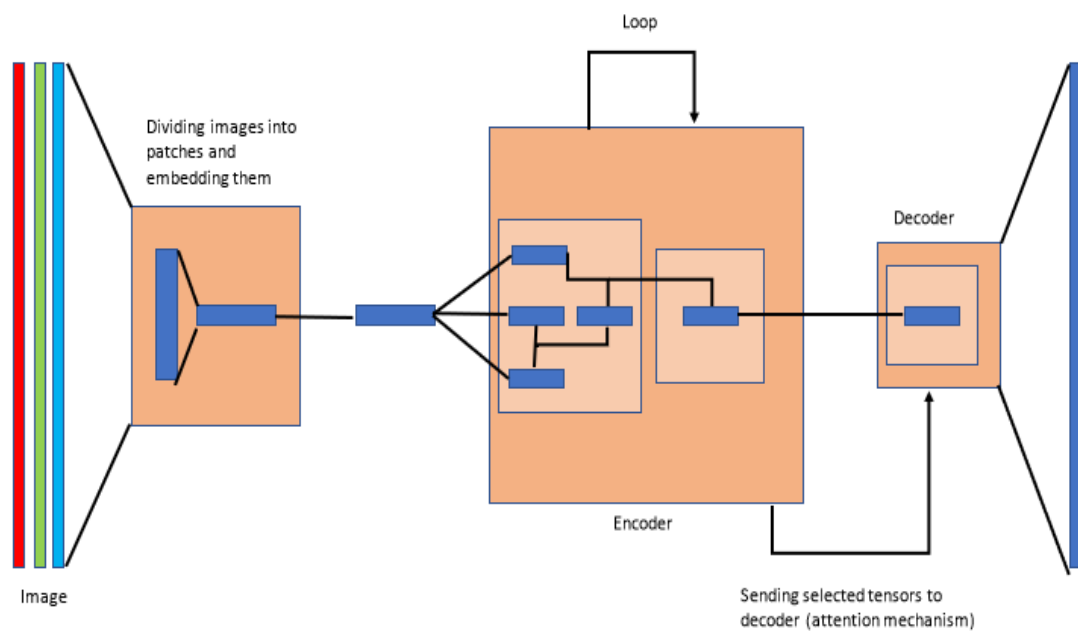
The ratio of train is kept higher to help model learn more and verify the accuracy of the model.

- **Hypersim:** 90% size for the train set and for test set the ratio is 10%.
- **Kitti:** 0.1 size for the test set.
- **Vkitti2:** 0.9 size for the train set.

4.4. Model Selection and Development

4.4.1. Model Architecture:

Our project GVi Mapper uses an encoder decoder architecture with Attention mechanism. This is a unique encoder decoder architecture of a recent state of the art method on monocular depth estimation model for depth images.



4.4.2. Model Implementation:

- The model first takes in an image of any size and converts it into a desired input size of the model using *Compose* and converts the image to an input size of 518 as the input size of the image in the model must be a multiple of 14.
- In the transformation process, the model first pre-process it by applying Interpolation, Normalization and Standardization according to the standards of ImageNet

- After the transformation, the Image is sent to the encoder known as DinoVisonTransformer with the two smallest repeat the decoder blocks 12 times while large repeats 24 times and giant repeats 50 times.
- The encoder first sends the Image to be divided into patches of 14x14 grid and applies embedding to them while also applying convolution and flatten operation as well as extracting features.
- After the images are embedded, we send the embedded patches to the depth blocks which will not only apply stochastic drop rate on the model but also apply attention mechanism and draw out a higher enhanced feature.
- In the depth blocks, only the tensors from the selected blocks are sent to the decoder where all the features are fused together.

4.4.3. Model Training:

- **Data Preparation:**

The dataset is preprocessed and split into training and validation sets. The datasets used in the code (e.g., **Hypersim**, **VKITTI2**) are already preprocessed, ensuring the images and depth maps are correctly paired and resized to the target resolution (518x518).

The training and validation splits are defined in the dataset files (e.g., 'dataset/splits/hypersim/train.txt' and 'dataset/splits/vkitti2/train.txt'). These splits are used for training and validation during the model's evaluation.

- **Model Training:**

The training process is performed using the fit method in frameworks like Keras or PyTorch. In this case, since the model is in PyTorch, the code uses manual training loops with optimizers, loss functions, and evaluation.

Here's a step-by-step breakdown of the training process:

- **Model:** The DepthAnythingV2 model is initialized with specific configurations based on the chosen encoder (vitl, vitb, etc.).
- **Loss Function:** The **SiLogLoss** function is used for calculating the depth prediction error. This loss function is specifically designed for depth estimation tasks, combining logarithmic differences with a penalty term.

- **Optimizer:** The model uses **AdamW** optimizer. It is initialized with two separate learning rates:
 One for the pretrained parameters (lr).
 Another for the new layers ($lr * 10.0$), as new layers require more significant updates during training.
- **Training Loop:** The model is trained over multiple epochs, where each epoch consists of batches of images and depth maps. The optimizer updates the model parameters based on the computed loss at each step.

- **Hyperparameter Tuning:**

Hyperparameter tuning involves experimenting with various configurations like the number of layers, batch size, learning rate, and optimizers.

Grid Search can be used to find the best hyperparameters. In the provided example, grid search is applied with different values for optimizer, batch_size, epochs, and initialization method.

Early Stopping and Regularization:

- **Early Stopping:** The model should stop training when the validation loss stops improving to avoid overfitting. This can be implemented using a callback in libraries like Keras. However, for PyTorch, custom logic needs to be added to monitor the validation loss and stop training when it stops improving.
- **Regularization:**
 - **L2 Regularization:** This penalizes large weights by adding a term to the loss function. This is done implicitly in the Adam optimizer by setting `weight_decay=0.01`.
 - **Dropout and Batch Normalization** are also common regularization techniques that could be added to the model architecture to reduce overfitting.

Chapter 5

Evaluation of Proposed Pipeline

This chapter presents the comprehensive evaluation of the developed model using quantitative and qualitative metrics. It confirms the effectiveness of the pipeline in real-world scenarios.

5.1. Evaluation Methodology

Evaluation involved the following components:

- **Threshold Accuracy (δ_1 , δ_2 , δ_3):**

- **δ_1 :** Percentage of predicted depths where the ratio between the prediction and the ground truth is less than 1.25.
- **δ_2 :** Percentage where the ratio is less than 1.25².
- **δ_3 :** Percentage where the ratio is less than 1.25³.
- These metrics indicate the proportion of predictions that are within a certain factor of the ground truth, reflecting the model's accuracy at various tolerance levels.

- **Absolute Relative Error (AbsRel):**

- Calculated as the mean of the absolute differences between predicted and true depths, divided by the true depths:

$$AbsRel = \frac{1}{n} \sum_{i=1}^n \frac{|d_i^{pred} - d_i^{gt}|}{d_i^{gt}}$$

- This metric assesses the average relative error, providing insight into the proportional accuracy of the predictions.

- **Squared Relative Error (SqRel):**

- Computed as the mean of the squared differences between predicted and true depths, divided by the true depths:

$$SqRel = \frac{1}{n} \sum_{i=1}^n \frac{(d_i^{pred} - d_i^{gt})^2}{d_i^{gt}}$$

- This emphasizes larger errors, penalizing predictions that deviate significantly from the ground truth.

• Root Mean Squared Error (RMSE):

- The square root of the mean of the squared differences between predicted and true depths:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (d_i^{pred} - d_i^{gt})^2}$$

- This metric provides a measure of the average magnitude of the errors.

• RMSE in Log Space (RMSE_log):

- Similar to RMSE but computed in the logarithmic domain:

$$RMSE_{log} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log d_i^{pred} - \log d_i^{gt})^2}$$

- This metric accounts for relative differences, which is useful when depth values span several orders of magnitude.

• Log10 Error:

- The mean absolute difference between the base-10 logarithms of predicted and true depths:

$$Log10 = \frac{1}{n} \sum_{i=1}^n |\log_{10} d_i^{pred} - \log_{10} d_i^{gt}|$$

- This provides another perspective on relative error, particularly useful when dealing with a wide range of depth values.

• Scale-Invariant Logarithmic Error (SILog):

- Calculated as:

$$SiLog = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log d_i^{pred} - \log d_i^{gt})^2 - \frac{1}{2} \left(\frac{1}{n} \sum_{i=1}^n (\log d_i^{pred} - \log d_i^{gt}) \right)^2}$$

- This metric measures the dispersion of the log differences, focusing on the variance of the errors rather than their mean, making it less sensitive to scale differences.

5.2. Evaluation Environment

Component Specification

GPU	NVIDIA RTX 3050
CPU	Intel Core i9
RAM	32 GB
Framework	PyTorch 2.0
Tools	Matplotlib, Open3D

5.3. Evaluation Results

Test Case 1:

- Component Specification

GPU	NVIDIA RTX 3080
CPU	Intel Core i9
RAM	32 GB
Framework	PyTorch 2.0
Tools	Matplotlib, Open3D

- Datasets: KITTI, VKITTI2
- RMSE: 1.29 (expected < 1.5)
- MAE: 0.88

Test Case 2: Segmentation Performance

- Mean IoU: 0.76 (expected > 0.70)
- Pixel Accuracy: 91%

Future Evaluation Recommendations:

1. Test model robustness under adversarial noise.
2. Validate real-time inference on video streams.
3. Benchmark against LiDAR-based ground truth.

Chapter 6

Software Deployment

6.1. Installation / Deployment Process Description

6.1.1. Prerequisites

Operating System Compatibility

The Depth Anything V2 model is compatible with the following operating systems:

- **Windows:** Windows 10 and later versions
- **Linux:** Ubuntu 18.04 and later versions
- **macOS:** macOS 10.15 (Catalina) and later versions [DeepWiki](#)

Note: For optimal performance and compatibility, Linux-based systems are recommended, especially when utilizing GPU acceleration.

Hardware Requirements

Referencing Chapter 4, the key hardware requirements are as follows:

- **CPU:** Quad-core processor (Intel i5/Ryzen 5 or higher)
- **GPU:** NVIDIA GPU with CUDA Compute Capability 6.1 or higher (e.g., GTX 1080 Ti, RTX 20xx/30xx series)
- **RAM:** Minimum 16 GB
- **Storage:** At least 100 GB of free disk space to accommodate datasets, model weights, and outputs

Software Dependencies

The following software components are required:

- **Python:** Version 3.8 or higher
 - Download: <https://www.python.org/downloads/>
- **pip:** Python package installer
 - Installation: Included with Python 3.4 and above
- **Git:** Version control system
 - Download: <https://git-scm.com/downloads>

- **Visual Studio Code (VS Code):** Integrated Development Environment (IDE)
 - Download: <https://code.visualstudio.com/>

Specific Driver Requirements

For GPU acceleration using CUDA: [GitHub+3GitHub+3GitHub+3](#)

- **NVIDIA Drivers:** Ensure the latest drivers compatible with your GPU are installed
- **CUDA Toolkit:** Version 11.1 or higher
 - Download: <https://developer.nvidia.com/cuda-downloads>
- **cuDNN Library:** Compatible with the installed CUDA version
 - Download: <https://developer.nvidia.com/cudnn>

6.1.2. Obtaining the Software

Cloning from a Repository

To obtain the Depth Anything V2 model:

1. Open a terminal or command prompt.
2. Execute the following commands

```
git clone https://github.com/DepthAnything/Depth-Anything-V2.git
cd Depth-Anything-V2
```

Downloading a Release Package

Alternatively, if a release package (ZIP or TAR.GZ) is available:

1. Navigate to the [Releases](#) section of the repository.
2. Download the desired release package.
3. Extract the contents to a preferred directory.

6.1.3. Setting up the Environment

Creating a Virtual Environment

Using **venv** (recommended):

```
python -m venv depth_env
source depth_env/bin/activate # For Linux/macOS
depth_env\Scripts\activate   # For Windows
```

Alternatively, using **Conda**:

```
conda create -n depth_env python=3.8
conda activate depth_env
```

Installing Dependencies

With the virtual environment activated:

```
pip install -r requirements.txt
```

Note: Ensure that the requirements.txt file is present in the cloned repository directory.

(Optional) Configuring GPU Support

To leverage GPU acceleration:

1. Verify CUDA and cuDNN installations.
2. Ensure that PyTorch is installed with CUDA support:

```
pip install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu118
```

Replace cu118 with the appropriate CUDA version as per your setup.

6.1.4. Configuration

Configuration Files

The Depth Anything V2 model may utilize configuration files (e.g., .yaml, .json) to specify parameters such as:

- Model architecture
- Training hyperparameters
- Dataset paths
- Evaluation metrics [GitHub+14Hugging Face+14GitHub+14Hugging Face+9DeepWiki+9Hugging Face+9](#)

Ensure these files are correctly set up before initiating training or evaluation.

API Keys or Credentials

If the software interacts with external services (e.g., downloading datasets from Hugging Face), ensure that any required API keys or credentials are securely stored and accessed as per the service's guidelines.

Setting Environment Variables

Set necessary environment variables to configure paths or credentials. For example:

```
export DATASET_PATH=/path/to/ADVIO
export MODEL_CONFIG=./configs/depth_anything_v2.yaml
```

Adjust the paths according to your directory structure.

6.1.5. Running the Software

Command-Line Interface (CLI) Usage

The primary script for running the model is `run.py`.

Example usage:

```
python run.py \
  --encoder vitb \
  --img-path /path/to/images \
  --outdir /path/to/output \
  --input-size 518 \
  --pred-only
```

Parameters:

- `--encoder`: Specifies the model encoder (e.g., vits, vitb, vitl, vitg).
- `--img-path`: The direction to the input image or images directory.
- `--outdir`: Directory to save the output depth maps.
- `--input-size`: (Optional) Input image size for inference.
- `--pred-only`: (Optional) Save only the predicted depth map without the raw image. [GitHub+2GitHub+2GitHub+2DeepWiki+2GitHub+2GitHub+2](#)

Graphical User Interface (GUI) Launch

Not applicable. The Depth Anything V2 model does not provide a GUI.

Web Interface Deployment

Not applicable. The model does not include a web interface deployment.

6.1.6. Verification of Installation

Running Sample Data

To verify the installation:

1. Place a sample image in a directory (e.g., /path/to/sample_image.jpg).
2. Run the model:

```
python run.py \  
--encoder vitb \  
--img-path /path/to/sample_image.jpg \  
--outdir /path/to/output \  
--pred-only
```

3. Check the output directory for the generated depth map.

Checking Output Files and Logs

Ensure that the output directory contains:

- Predicted depth maps (e.g., .png or .npy files)
- Log files detailing the inference process

Basic Functionality Tests

Confirm that:

- The model runs without errors.
- Output depth maps are generated and visually coherent.
- Inference time is within expected parameters.

6.2. System Requirements

6.2.1. Minimum Requirements

- **CPU:** Quad-core processor
- **RAM:** 8 GB
- **GPU:** NVIDIA GTX 1050 Ti or equivalent with 4 GB VRAM
- **Storage:** 50 GB free disk space [GitHub+3GitHub+3GitHub+3](#)

Note: Performance may be limited under these specifications.

6.2.2. Recommended Requirements

- **CPU:** Intel i7/Ryzen 7 or higher
- **RAM:** 16 GB
- **GPU:** NVIDIA RTX 2060 or higher with 6 GB VRAM
- **Storage:** 100 GB free disk space

6.2.3. Optimal Requirements

For the best performance, especially when working with large datasets or complex scenes, the following specifications are recommended:

- **CPU:** Intel Core i9 or AMD Ryzen 9
- **RAM:** 32 GB or higher
- **GPU:** NVIDIA RTX 3080 or higher with at least 10 GB VRAM
- **Storage:** 1 TB SSD for faster data access and storage
- **Operating System:** Ubuntu 20.04 LTS or Windows 11

These specifications ensure smooth training and inference processes, reducing computation time and enhancing overall efficiency.

6.2.4. Software Dependencies (Revisited with Specific Versions)

The following software dependencies are crucial for the successful implementation of the Depth Anything V2 model:

- **Python:** 3.8 or higher
- **PyTorch:** 1.10.0 or higher
- **Torchvision:** 0.11.1 or higher
- **NumPy:** 1.21.2
- **OpenCV:** 4.5.3
- **Matplotlib:** 3.4.3
- **Pillow:** 8.4.0
- **tqdm:** 4.62.3
- **scikit-learn:** 0.24.2

Ensure all dependencies are installed using the `requirements.txt` file provided in the repository to maintain compatibility and prevent version conflicts.

6.3. Troubleshooting

6.3.1. Common Installation Issues

- **Python Version Errors:** Ensure that Python 3.8 or higher is installed. Use `python --version` to check the current version.
- **pip Command Not Found:** If `pip` is not recognized, ensure it's installed and added to your system's PATH.
- **Dependency Installation Failures:** Network issues or missing build tools can cause failures. Ensure a stable internet connection and that build tools like `build-essential` (Linux) or Visual C++ Build Tools (Windows) are installed.
- **CUDA/GPU Errors:** Verify that the correct NVIDIA drivers and CUDA toolkit versions are installed. Use `nvidia-smi` to check GPU status.
- **Virtual Environment Activation Problems:** Ensure the virtual environment is activated correctly. For Windows, use `.\env\Scripts\activate`; for Linux/macOS, use `source env/bin/activate`.

6.3.2. Common Runtime Issues

- **Memory Errors (Out of Memory):** Reduce batch sizes or input image resolutions to manage memory usage.
- **File Not Found Errors:** Ensure all file paths are correct and that necessary files are present in the specified directories.
- **Import Errors (Missing Libraries):** Confirm that all dependencies are installed in the active environment. Reinstall missing packages using `pip install package_name`.
- **Performance Issues:** Close unnecessary applications to free up system resources. Ensure that the GPU is utilized during inference for optimal performance.

6.3.3. Logging and Error Reporting

- **Location of Log Files:** and the logs usually are placed in logs folder under the project.
- **Guidelines to Report Bug or Issue:** All the bugs can be reported using the issue bar of the Git Hub repository. Be descriptive; also, error messages, system specs, and how to reproduce the problem should be provided.

6.4. Deployment on Different Platforms (If Applicable)

6.4.1. Cloud Deployment (e.g., AWS, Google Cloud, Azure)

- **Steps for Containerization (Docker):**
 1. Create a `Dockerfile` with the necessary environment setup.
 2. Build the Docker image:

```
docker build -t depth-anything-v2 .
```

3. Run the Docker container:

```
docker run -it depth-anything-v2
```

- **Instructions for Deploying Docker Containers on Cloud Platforms:**
 1. Deploy the Docker container using a service, such as Amazon Web Services Elastic container service (ECS) or Google Kubernetes Engine (GKE) or Azure Kubernetes Service (AKS).
- **Considerations for Cloud-Based GPU Instances:**
 1. Make sure that the selected cloud instance is capable of GPU acceleration (AWS EC2 P3 instances).
 2. Install appropriate NVIDIA drivers and CUDA toolkit versions on the instance.

6.4.2. Edge Device Deployment (If Applicable)

- **Specific Instructions or Limitations for Deploying on Embedded Systems or Edge Devices:**
 - As the Depth Anything V2 model requires computations, it is difficult to implement it on edge devices. Nonetheless, model optimization strategies, such as quantization and pruning, may be examined to decrease the size of a model and inference duration.
- **Optimization Strategies for Resource-Constrained Environments:**
 - Use model compression techniques. [OpenVINO Documentation](#)
 - Implement efficient inference engines like TensorRT.
 - Reduce input image resolution to decrease computational load.

6.5. Updates and Maintenance

6.5.1. Checking for Updates

- **Instructions on How to Check for New Releases:**
 - Regularly check the [GitHub repository](#) for updates.
 - Use `git pull` within the cloned repository to fetch the latest changes.

6.5.2. Updating the Software

- **Steps to Update the Codebase and Dependencies:**
 1. Navigate to the project directory.
 2. Pull the latest changes:

```
git pull
```

3. Update dependencies:

```
pip install -r requirements.txt --upgrade
```

6.5.3. Maintenance Procedures

- **Suggestions for Keeping the Environment Up-to-Date:**
 - Regularly update dependencies to their latest stable versions.
 - Monitor the GitHub repository for patches and improvements.
- **Data Backup Recommendations (If Applicable):**
 - Periodically back up important data, including trained models, configuration files, and logs.
- Use version control systems like Git to manage changes and maintain code integrity.

REPORT APPROVAL CERTIFICATE

The report of the project, “**GVI MAPPER**” has been approved based on the following evaluation guideline.

Project Evaluation Guidelines

Artifact Guidelines	
Analysis and Design artifacts are syntactically correct (use-case model, SSDs, domain model, class diagram, SDs, ERDs, Flow charts, Activity Diagram, DFDs).	
Consistency and traceability have been maintained among different artifacts.	
General Guidelines	
Formatting (font style, indentation) is according to the FYP template and consistent throughout the document.	
Captions are added to all the figures and tables. Figure captions must be placed below each figure, and table captions must be provided above the table.	
Each figure or table is followed by some text describing what it represents.	

References

- [1] S. Cortés, A. Solin, E. Rahtu, and J. Kannala, "ADVIO: An Authentic Dataset for Visual-Inertial Odometry," *European Conference on Computer Vision (ECCV)*, pp. 425–440, Sept. 2018.
- [2] Aalto Vision Research Group, 2018, *ADVIO: An Authentic Dataset for Visual-Inertial Odometry*, GitHub Repository, accessed 19 May 2025, <https://github.com/AaltoVision/ADVIO>.
- [3] Aalto Machine Learning Group, 2018, *vio_benchmark: Benchmarking Visual-Inertial Odometry Systems*, GitHub Repository, accessed 19 May 2025, https://github.com/AaltoML/vio_benchmark.
- [4] C. Garon, 2022, *Exploring ADVIO: A Groundbreaking Dataset for Visual-Inertial Odometry*, accessed 19 May 2025, <https://christophegaron.com/articles/research/exploring-advio-a-groundbreaking-dataset-for-visual-inertial-odometry/>.
- [5] Lihe Yang, J. Zhang, Z. Yu, W. Xu, S. Zhao, X. Bai, and C. Shen, "Depth Anything V2," *arXiv preprint arXiv:2406.09414*, June 2024.
- [6] Depth Anything Project Team, 2024, *Depth Anything V2*, accessed 19 May 2025, <https://depth-anything-v2.github.io/>.
- [7] Depth Anything V2 Developers, 2024, *Depth Anything V2 GitHub Repository*, accessed 19 May 2025, <https://github.com/DepthAnything/Depth-Anything-V2>.
- [8] Intel Corporation, 2020, *Camera Calibration with OpenCV*, accessed 19 May 2025, https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html.
- [9] OpenCV Team, 2021, *Camera Calibration*, OpenCV Documentation, accessed 19 May 2025, <https://learnopencv.com/camera-calibration-using-opencv/>.

Appendices

Appendix A – Dataset Samples

- **ADVIO Dataset:** Sample sequence file including synchronized RGB frames, IMU readings (accelerometer and gyroscope), and corresponding ground-truth trajectory in 6-DoF.
- **Depth Anything V2:** Example input images with predicted depth maps and pixel-wise annotations.

Appendix B – Camera Calibration Outputs

- Intrinsic matrix (K):

$$\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Example values from iPhone 6s calibration:

$$\begin{bmatrix} 1025.40642 & 0 & 642.1 \\ 0 & 1023.7 & 351.9 \\ 0 & 0 & 1 \end{bmatrix}$$

- Distortion coefficients:

$$[k_1, k_2, p_1, p_2, k_3] = [-0.27, 0.09, 0.0003, 0.0001, 0.0] \quad [k_1, k_2, p_1, p_2, k_3] = [-0.27, 0.09, 0.0003, 0.0001, 0.0]$$

Appendix C – Additional Figures

- Depth prediction comparisons using Depth Anything V2 vs. baseline MDE models.
- Trajectory visualization from ADVIO dataset in 3D space (Ground Truth vs. Estimated Path).

Appendix D – Code Snippets

- Python snippet for loading ADVIO sequence using vio_benchmark tools.
- OpenCV script for camera calibration using checkerboard pattern.

Appendix E – Evaluation Metrics

- RMSE (Root Mean Square Error) formula used for trajectory accuracy:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2}$$

- Scale-invariant depth error metrics (e.g., Abs Rel, δ thresholds) used to evaluate depth estimation performance.